

МИНОБРАЗОВАНИЯ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра цифровых интеллектуальных систем

*Разработка интеллектуальной системы учета, анализа и прогнозирования
личных финансов на основе Mini App и Telegram-бота*

Бакалаврская работа
09.03.02 Информационные системы и технологии
Профиль Информационные технологии в цифровом дизайне

Зав. кафедрой  М. И. Чижов

д.т.н., профессор 01.06.2026

Обучающийся  В. С. Гаврилов

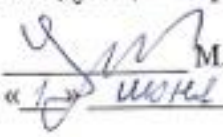
Руководитель  А. И. Чекмарев

ст. преподаватель

Воронеж 2026

МИНОБРАЗОВАНИЯ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра цифровых интеллектуальных систем

УТВЕРЖДАЮ
Заведующий кафедрой
 М. И. Чижов
« 1 » июня 2026 г.

ЗАДАНИЕ
НА ВЫПОЛНЕНИЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
ОБУЧАЮЩЕГОСЯ Гаврилова Владислава Сергеевича

1. Тема работы «Разработка интеллектуальной системы учета, анализа и прогнозирования личных финансов на основе Mini App и Telegram-бота», утверждена решением ученого совета факультета компьютерных наук от 08.10.2025 г.
2. Направление подготовки: 09.03.02 Информационные системы и технологии
3. Срок сдачи законченной работы: __. __.20__
4. Календарный план:

№	Структура ВКР	Сроки выполнения	Примечание
1	Введение	09.10.2025 – 15.10.2025	
2	1 Постановка задачи и анализ предметной области	16.10.2025 – 20.11.2025	
3	1.1 Актуальность разработки системы	16.10.2025 – 22.10.2025	
4	1.2 Анализ предметной области	23.10.2025 – 29.10.2025	
5	1.3 Обзор существующих решений	30.10.2025 – 05.11.2025	
6	1.4 Сравнительный анализ аналогов	06.11.2025 – 10.11.2025	
7	1.5 Постановка задачи разработки	11.11.2025 – 14.11.2025	
8	1.6 Требования к системе	15.11.2025 – 18.11.2025	
9	1.7 Выводы по разделу	19.11.2025 – 20.11.2025	
10	2 Проектирование системы	21.11.2025 – 20.12.2025	
11	2.1 Общая архитектура системы	21.11.2025 – 25.11.2025	
12	2.2 Проектирование базы данных	26.11.2025 – 30.11.2025	
13	2.3 Проектирование Telegram-бота	01.12.2025 – 04.12.2025	
14	2.4 Проектирование Mini App	05.12.2025 – 08.12.2025	
15	2.5 Проектирование REST API	09.12.2025 – 12.12.2025	
16	2.6 Проектирование аналитики	13.12.2025 – 15.12.2025	
17	2.7 Проектирование модуля машинного обучения	16.12.2025 – 18.12.2025	
18	2.8 Проектирование дополнительных модулей	19.12.2025 – 19.12.2025	
19	2.9 Выводы по разделу	20.12.2025 – 20.12.2025	
20	3 Реализация программного обеспечения	21.12.2025 – 15.04.2026	
21	3.1 Выбор средств разработки	21.12.2025 – 25.12.2025	
22	3.2 Реализация базы данных	26.12.2025 – 10.01.2026	
23	3.3 Структура программного проекта	11.01.2026 – 20.01.2026	
24	3.4 Реализация Telegram-бота	21.01.2026 – 15.02.2026	
25	3.5 Реализация Mini App	16.02.2026 – 10.03.2026	

26	3.6 Реализация REST API	11.03.2026 – 25.03.2026	
27	3.7 Реализация аналитики и отчетов	26.03.2026 – 02.04.2026	
28	3.8 Реализация машинного обучения и прогнозирования	03.04.2026 – 10.04.2026	
29	3.9 Реализация дополнительных функций	11.04.2026 – 14.04.2026	
30	3.10 Выводы по разделу	15.04.2026 – 15.04.2026	
31	4 Тестирование и оценка результатов	16.04.2026 – 20.05.2026	
32	4.1 Цель и методы тестирования	16.04.2026 – 18.04.2026	
33	4.2 Тестирование Telegram-бота	19.04.2026 – 24.04.2026	
34	4.3 Тестирование Mini App	25.04.2026 – 30.04.2026	
35	4.4 Тестирование REST API	01.05.2026 – 05.05.2026	
36	4.5 Тестирование аналитики	06.05.2026 – 09.05.2026	
37	4.6 Тестирование ML-категоризации и прогнозирования	10.05.2026 – 14.05.2026	
38	4.7 Тестирование импорта, экспорта и дополнительных функций	15.05.2026 – 17.05.2026	
39	4.8 Анализ результатов тестирования	18.05.2026 – 19.05.2026	
40	4.9 Выводы по разделу	20.05.2026 – 20.05.2026	
41	Заключение	21.05.2026 – 25.05.2026	
42	Список использованных источников	26.05.2026 – 29.05.2026	
43	Оформление выпускной квалификационной работы	04.06.2026 – 10.06.2026	

Обучающийся



Гаврилов В. С.

Руководитель



Чекмарев А. И.

РЕФЕРАТ

Выпускная квалификационная работа содержит 95 с., 52 рис., 9 табл., 23 использованных источника, 0 приложений.

ЛИЧНЫЕ ФИНАНСЫ, TELEGRAM-БОТ, TELEGRAM MINI APP, REST API, POSTGRESQL, АНАЛИТИКА, БЮДЖЕТЫ, ИМПОРТ ВЫПИСОК, ЭКСПОРТ ОТЧЕТОВ, МАШИННОЕ ОБУЧЕНИЕ, КАТЕГОРИЗАЦИЯ РАСХОДОВ, ПРОГНОЗИРОВАНИЕ РАСХОДОВ

Объектом разработки является система учета, анализа и прогнозирования личных финансов.

Целью работы является разработка интеллектуальной системы учета личных финансов на основе Telegram-бота и Telegram Mini App с применением методов машинного обучения.

В процессе выполнения работы были проанализированы существующие решения в области учета личных финансов, сформулированы требования к системе, спроектированы архитектура, база данных, Telegram-бот, Mini App, REST API, аналитические модули и модуль машинного обучения. Также была выполнена программная реализация системы и проведено тестирование основных пользовательских сценариев.

Результатом работы стала программная система, включающая Telegram-бот для быстрого ввода операций, Mini App для расширенной работы с данными, серверную часть, REST API, базу данных PostgreSQL, модуль аналитики, импорт банковских выписок, экспорт отчетов и модуль машинного обучения для категоризации и прогнозирования расходов.

Практическая значимость работы заключается в том, что система может использоваться для повседневного учета доходов и расходов, анализа финансовой активности, контроля бюджета и планирования будущих трат.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	7
ВВЕДЕНИЕ	8
1 Постановка задачи и анализ предметной области	11
1.1 Актуальность разработки системы	11
1.2 Анализ предметной области	12
1.3 Обзор существующих решений	13
1.4 Сравнительный анализ аналогов	17
1.5 Постановка задачи разработки	19
1.6 Требования к системе	19
1.7 Выводы по разделу	22
2 Проектирование системы	23
2.1 Общая архитектура системы	23
2.2 Проектирование базы данных	24
2.3 Проектирование Telegram-бота	25
2.4 Проектирование Mini App	28
2.5 Проектирование REST API	30
2.6 Проектирование аналитики	32
2.7 Проектирование модуля машинного обучения	33
2.8 Проектирование дополнительных модулей	36
2.9 Выводы по разделу	38
3 Реализация программного обеспечения	39
3.1 Выбор средств разработки	39
3.2 Реализация базы данных	39
3.3 Структура программного проекта	41
3.4 Реализация Telegram-бота	43
3.5 Реализация Mini App	54
3.6 Реализация REST API	70
3.7 Реализация аналитики и отчетов	74
3.8 Реализация машинного обучения и прогнозирования	75
3.9 Реализация дополнительных функций	78

3.10 Выводы по разделу	79
4 Тестирование и оценка результатов	81
4.1 Цель и методы тестирования.....	81
4.2 Тестирование Telegram-бота	81
4.3 Тестирование Mini App.....	83
4.4 Тестирование REST API	84
4.5 Тестирование аналитики.....	85
4.6 Тестирование ML-категоризации и прогнозирования	86
4.7 Тестирование импорта, экспорта и дополнительных функций	87
4.8 Анализ результатов тестирования.....	88
4.9 Выводы по разделу	89
ЗАКЛЮЧЕНИЕ	90
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	93

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей выпускной квалификационной работе применяются следующие термины с соответствующими определениями:

Telegram-бот — программный компонент, который взаимодействует с пользователем через сообщения, команды и кнопки в Telegram.

Telegram Mini App — встроенное веб-приложение, открываемое внутри Telegram и используемое для отображения расширенного интерфейса системы [3].

REST API — программный интерфейс, через который клиентская часть обменивается данными с сервером с использованием HTTP-запросов [8], [9].

PostgreSQL — реляционная система управления базами данных, используемая для хранения пользователей, категорий, транзакций, бюджетов и других данных системы [6].

FSM — конечный автомат, применяемый для пошаговых сценариев работы Telegram-бота.

ML — машинное обучение, используемое в системе для автоматической категоризации расходов и прогнозирования будущих трат [10], [21].

Транзакция — финансовая операция пользователя, отражающая доход или расход.

Категория — группа, к которой относится финансовая операция.

Бюджет — установленный пользователем лимит расходов за выбранный период.

Импорт выписки — загрузка банковского документа с последующим извлечением операций.

Экспорт отчета — выгрузка финансовых данных пользователя во внешний файл.

ВВЕДЕНИЕ

Ведение личных финансов является важной задачей для пользователя, который хочет контролировать доходы, расходы и планировать будущие траты. На практике многие люди не ведут учет регулярно или используют для этого различные инструменты: заметки, таблицы, банковские приложения или простые мобильные сервисы. Такой подход не всегда удобен, так как данные приходится вносить вручную, а для анализа часто нужно дополнительно обрабатывать информацию.

Одной из главных проблем является быстрый ввод операций. Если пользователь не записывает расход сразу, он может забыть сумму, дату или назначение платежа. Поэтому система учета должна быть максимально простой в повседневном использовании. Для этой цели удобно использовать Telegram-бота, так как Telegram уже является привычной средой общения, а Bot API позволяет создавать программные интерфейсы для обработки сообщений и команд пользователя [2].

При этом одного текстового интерфейса недостаточно. Через Telegram-бота удобно быстро добавить расход, посмотреть баланс или открыть меню, но для просмотра аналитики, графиков, бюджетов и истории операций лучше подходит графический интерфейс. Поэтому в работе используется Telegram Mini App — встроенное веб-приложение, открываемое внутри Telegram [3].

Актуальность темы заключается в разработке единой системы, которая объединяет быстрый ввод операций через Telegram-бота, расширенный интерфейс Mini App, хранение данных в PostgreSQL, аналитику, бюджетирование и элементы машинного обучения. Такая система должна не только сохранять доходы и расходы, но и помогать пользователю анализировать финансовую активность и планировать будущие траты.

Отдельное значение имеет применение машинного обучения. В обычных системах пользователь часто сам выбирает категорию каждой операции. Это занимает время и может приводить к ошибкам. В

разрабатываемой системе описание операции и сумма используются для автоматического определения категории расхода. Для обработки текстовых данных применяется TF-IDF-векторизация, а для классификации может использоваться логистическая регрессия [10], [11], [12]. Также система формирует прогноз будущих расходов на основе истории транзакций.

Объектом работы является система учета, анализа и прогнозирования личных финансов.

Предметом работы являются методы и средства проектирования и реализации программной системы, включающей Telegram-бот, Mini App, REST API, базу данных PostgreSQL, модуль аналитики и модуль машинного обучения.

Целью выпускной квалификационной работы является разработка интеллектуальной системы учета, анализа и прогнозирования личных финансов на основе Telegram-бота и Mini App с применением методов машинного обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область учета личных финансов и существующие решения;
- сформулировать требования к разрабатываемой системе;
- спроектировать архитектуру системы, базу данных, Telegram-бот, Mini App и REST API;
- реализовать серверную часть, Telegram-бота и Mini App;
- реализовать хранение данных в PostgreSQL;
- реализовать функции добавления, редактирования, удаления и фильтрации операций;
- реализовать аналитику, бюджеты, импорт выписок и экспорт отчетов;
- реализовать автоматическую категоризацию расходов и прогнозирование;

- провести тестирование системы и оценить результаты работы.

В качестве основных средств разработки используются Python, aiogram, Django, Django REST Framework, PostgreSQL, HTML, CSS, JavaScript, Chart.js, scikit-learn, Matplotlib, openpyxl, XlsxWriter и pdfplumber. Python применяется для реализации серверной логики, Telegram-бота, аналитических и ML-модулей. Django и Django REST Framework используются для серверной части и REST API. PostgreSQL применяется как основная база данных. Mini App реализован как веб-интерфейс, а для построения графиков в интерфейсе используется Chart.js. Scikit-learn применяется для автоматической категоризации расходов, а библиотеки openpyxl, XlsxWriter и pdfplumber используются для экспорта отчетов и обработки банковских выписок [4], [5], [6], [7], [8], [10], [13], [14], [15], [16].

Практическая значимость работы заключается в том, что разработанная система может использоваться как инструмент для повседневного учета личных финансов. Пользователь получает возможность быстро добавлять операции через Telegram-бота, работать с аналитикой в Mini App, контролировать бюджеты, импортировать банковские выписки, выгружать отчеты и получать прогноз расходов.

Выпускная квалификационная работа состоит из введения, четырех разделов, заключения и списка использованных источников. В первом разделе рассматриваются постановка задачи и анализ предметной области. Во втором разделе выполняется проектирование системы. В третьем разделе описывается реализация программного обеспечения. В четвертом разделе приводятся тестирование и оценка результатов.

1 Постановка задачи и анализ предметной области

1.1 Актуальность разработки системы

Учет личных финансов нужен для контроля доходов, расходов и планирования бюджета. На практике многие пользователи ведут учет нерегулярно или используют для этого заметки, таблицы, банковские приложения и отдельные сервисы. Такие способы не всегда удобны, так как требуют ручного ввода, самостоятельного анализа и постоянного перехода между разными инструментами.

Одной из главных проблем является быстрый ввод операций. Если пользователь не записал расход сразу, он может забыть сумму, дату или назначение платежа. Поэтому система учета должна быть простой и доступной в повседневном использовании.

Удобным решением является Telegram-бот, так как Telegram используется пользователем ежедневно, а Telegram Bot API позволяет создавать ботов для обработки сообщений, команд и кнопок [2]. Через бота удобно быстро добавить доход или расход, посмотреть баланс и выполнить простые действия.

Однако для полноценного анализа одного бота недостаточно. Просматривать графики, бюджеты, историю операций и прогнозы удобнее в графическом интерфейсе. Поэтому в системе используется Telegram Mini App, который открывается внутри Telegram и позволяет работать с данными в виде полноценного веб-интерфейса [3].

Актуальность разработки заключается в создании единой системы, которая объединяет Telegram-бот, Mini App, базу данных PostgreSQL, REST API, аналитику и машинное обучение. Такая система должна не только хранить операции, но и помогать пользователю анализировать расходы, контролировать бюджет и прогнозировать будущие траты.

1.2 Анализ предметной области

Предметная область учета личных финансов включает ввод, хранение и обработку сведений о доходах, расходах, категориях, бюджетах и истории операций. В рамках данной работы она рассматривается не только как ведение списка транзакций, но и как система, которая помогает пользователю анализировать финансовые данные и планировать будущие траты.

Основной единицей учета является транзакция. Она содержит тип операции, сумму, дату, время, категорию и комментарий. Тип операции позволяет разделять доходы и расходы, сумма используется для расчета баланса, а дата и время нужны для фильтрации истории и построения аналитики за выбранный период. Хранение таких данных выполняется в базе PostgreSQL, которая подходит для организации связанных таблиц пользователей, категорий, транзакций и бюджетов [6].

Категории используются для группировки финансовых операций. Благодаря этому пользователь может видеть не только общую сумму расходов, но и основные направления трат: продукты, транспорт, маркетплейсы, образование, коммунальные платежи и другие категории. Эти данные применяются при формировании истории операций, построении диаграмм, контроле бюджета и подготовке отчетов.

Бюджеты нужны для ограничения расходов за выбранный период. Пользователь может задать лимит по определенной категории, после чего система сравнивает фактические траты с установленным значением. Такой механизм позволяет использовать приложение не только для учета, но и для контроля финансового поведения.

Аналитика в системе строится на основе сохраненных транзакций. Она включает расчет доходов, расходов, баланса, распределение операций по категориям и динамику трат по дням или месяцам. Для быстрого ввода операций используется Telegram-бот, а для более подробного просмотра

истории, графиков и бюджетов применяется Telegram Mini App, открываемый внутри Telegram [3].

Отдельное значение имеет машинное обучение. В обычных системах пользователь сам выбирает категорию каждой операции, что занимает время и может приводить к ошибкам. В разрабатываемой системе категория расхода может определяться автоматически по описанию операции и сумме. Для обработки текстового описания применяется TF-IDF-векторизация [11], а для классификации используется логистическая регрессия [12]. Данные методы реализуются средствами библиотеки scikit-learn [10].

Также система должна выполнять прогнозирование расходов на основе истории операций пользователя. Такой прогноз помогает заранее оценить возможные траты за следующий период и лучше планировать бюджет. Таким образом, предметная область разрабатываемой системы включает не только хранение финансовых операций, но и их анализ, категоризацию, бюджетирование и прогнозирование.

1.3 Обзор существующих решений

Для учета личных финансов используются разные подходы: электронные таблицы, банковские приложения, отдельные мобильные приложения и Telegram-боты. Каждый из этих вариантов решает часть задачи, однако не всегда закрывает все потребности пользователя в быстром вводе, аналитике, импорте данных и прогнозировании.

Электронные таблицы позволяют самостоятельно настроить структуру учета, добавить формулы, фильтры и диаграммы. Такой способ удобен для пользователей, которым важно полностью контролировать формат данных. Однако таблицы требуют регулярного ручного заполнения, самостоятельной настройки расчетов и отдельного открытия файла. Поэтому при повседневном использовании пользователь может быстро перестать вносить операции сразу после покупки.

Банковские приложения частично решают проблему ручного ввода, так как операции по картам отображаются автоматически. При этом они обычно ограничены данными конкретного банка и не всегда позволяют удобно учитывать наличные расходы, операции из других банков, собственные категории и пользовательские комментарии. Кроме того, аналитика в банковских приложениях чаще всего строится по внутренним правилам банка и не всегда может быть адаптирована под индивидуальную структуру учета.

Мобильные приложения для учета финансов обычно имеют более развитый интерфейс, графики и настройки категорий. Их недостаток заключается в необходимости устанавливать и регулярно открывать отдельное приложение. Для пользователя, который уже активно использует Telegram, дополнительный переход в другое приложение может снижать регулярность учета.

Telegram-боты удобны тем, что позволяют добавлять операции прямо в привычной среде общения. Пользователю достаточно отправить сообщение или нажать кнопку, а обработка команды выполняется через Telegram Bot API [2]. Такой подход хорошо подходит для быстрого ввода доходов и расходов. Однако простые Telegram-боты часто ограничиваются фиксацией операций и не имеют расширенного интерфейса, собственной серверной части, REST API, импорта выписок, прогнозирования и полноценной аналитики. Для решения этой проблемы можно использовать Telegram Mini App, которое открывается внутри Telegram и позволяет реализовать более удобный графический интерфейс [3].

В качестве ближайших аналогов были рассмотрены Telegram-боты @cointrybot и @mobs_bot. Они выбраны для сравнения, так как работают в той же пользовательской среде и предназначены для учета доходов и расходов через Telegram.

1.3.1 Telegram-бот @cointrybot

Telegram-бот @cointrybot предназначен для быстрого ввода финансовых операций через сообщения. После запуска бот показывает инструкцию, где объясняется, как добавить расход или доход. Расход вводится суммой и описанием, а доход — с использованием знака «+».

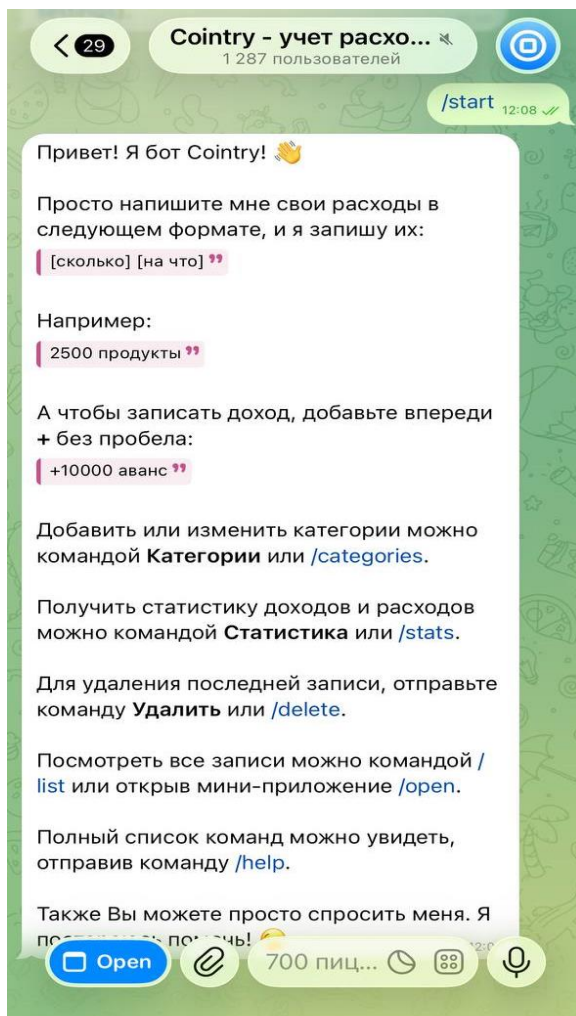


Рисунок 1 – Интерфейс Telegram-бота @cointrybot

На рисунке 1 показан интерфейс Telegram-бота @cointrybot. Видно, что бот делает акцент на быстром текстовом вводе и использовании коротких команд.

Преимуществом данного решения является простой старт. Пользователь сразу видит пример ввода операции и может быстро начать работу. Недостаток состоит в том, что система больше ориентирована на фиксацию операций, чем на полноценную аналитику, бюджетирование и прогнозирование.

1.3.2 Telegram-бот @mobs_bot

Telegram-бот @mobs_bot также предназначен для учета личных финансов, но использует связку с Google Таблицами. После запуска пользователь получает ссылку на таблицу, видит статус и базовую валюту. Операции вводятся через сообщения в Telegram.

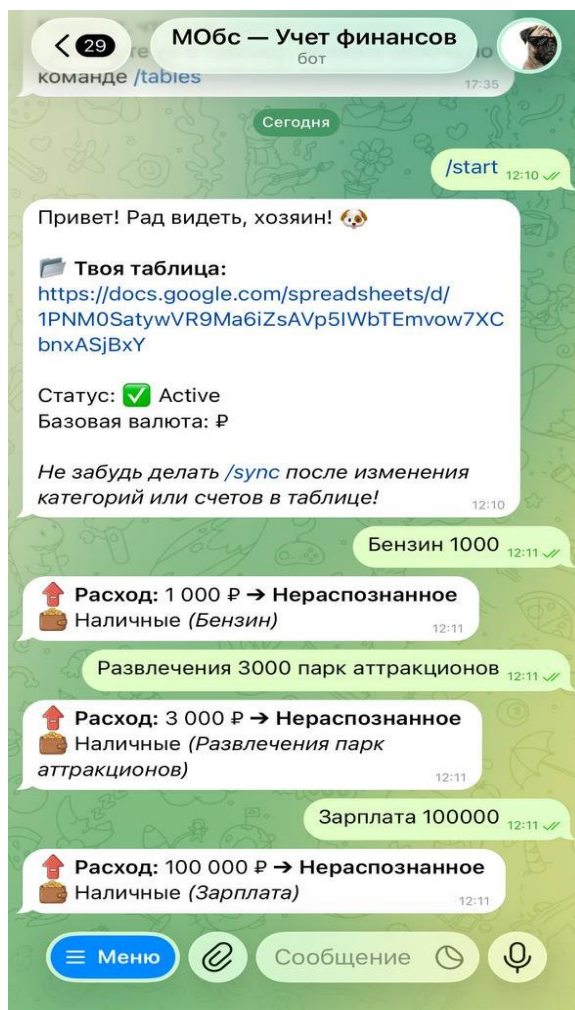


Рисунок 2 – Интерфейс Telegram-бота @mobs_bot

На рисунке 2 показан интерфейс Telegram-бота @mobs_bot. Видно, что данные связаны с внешней таблицей, а операции вводятся через текстовые сообщения.

Преимуществом такого подхода является возможность просмотра данных в табличном виде. Однако у решения есть недостатки: зависимость от

Google Таблиц, необходимость синхронизации и не всегда точное распознавание пользовательского ввода.

1.3.3 Разрабатываемая система @fffinansial_bot

Разрабатываемая система @fffinansial_bot отличается тем, что представляет собой не только Telegram-бот, а полноценную систему учета, анализа и прогнозирования личных финансов.

В состав системы входят Telegram-бот, Mini App, серверная часть, REST API, база данных PostgreSQL, аналитика, импорт выписок, экспорт отчетов, бюджеты, шаблоны, напоминания и модуль машинного обучения.

Telegram-бот используется для быстрого добавления операций, просмотра баланса, истории, импорта, экспорта и напоминаний. Mini App применяется для более удобной работы с транзакциями, графиками, бюджетами, категориями и прогнозами. Серверная часть и REST API обеспечивают централизованную обработку данных [8], [9].

Главным отличием системы является наличие интеллектуальных функций. При добавлении расхода система может предложить категорию автоматически, а также сформировать прогноз будущих расходов на основе истории операций.

1.4 Сравнительный анализ аналогов

Для сравнения рассмотренных решений была составлена таблица 1.

Таблица 1 – Сравнительный анализ существующих решений

Критерий	@cointrybot	@mobs_bot	@fffinansial_bot
Работа в Telegram	Да	Да	Да
Быстрый ввод операций	Да	Да	Да
Понятный стартовый сценарий	Да	Частично	Да

Mini App	Да	Нет	Да
Расширенный интерфейс	Ограниченно	Нет	Да
Собственная база данных	Неочевидно	Нет	Да
Зависимость от внешних сервисов	Неочевидно	Да	Нет
REST API	Нет данных	Нет данных	Да
Аналитика и графики	Базовые	Ограниченные	Да
Бюджеты	Неочевидно	Нет	Да
Импорт банковских выписок	Нет данных	Нет	Да
Экспорт отчетов	Ограниченно	Ограниченно	Да
Напоминания	Нет данных	Нет	Да
Шаблоны операций	Нет данных	Нет	Да
Автоматическая категоризация	Нет данных	Ограниченно	Да
Прогнозирование расходов	Нет	Нет	Да

Из таблицы видно, что рассмотренные аналоги в основном решают задачу быстрого ввода операций, но не предоставляют полноценную систему для комплексного учета, анализа и прогнозирования личных финансов. Их ограничения связаны с отсутствием развитого графического интерфейса, собственной серверной логики, полноценного REST API, расширенной аналитики, прогнозирования и гибкой работы с данными.

Разрабатываемая система @fffinansial_bot отличается тем, что объединяет несколько компонентов в единую архитектуру: Telegram-бот используется для быстрого ввода операций, Mini App — для удобной работы с историей, графиками, бюджетами и прогнозами, серверная часть — для обработки бизнес-логики, PostgreSQL — для централизованного хранения данных, а REST API — для связи между интерфейсами и сервером. Дополнительным отличием является наличие ML-модуля, который помогает автоматически определять категории расходов и снижает количество ручных действий пользователя.

Таким образом, разрабатываемая система решает задачу учета личных финансов более комплексно, чем рассмотренные аналоги, так как сочетает

быстрый ввод, визуальную аналитику, импорт и экспорт данных, бюджетирование, административные функции и элементы машинного обучения.

1.5 Постановка задачи разработки

На основе анализа предметной области и существующих решений была сформулирована задача разработки интеллектуальной системы учета, анализа и прогнозирования личных финансов.

Система должна позволять пользователю:

- быстро добавлять доходы и расходы через Telegram-бота;
- работать с транзакциями через Mini App;
- хранить данные в PostgreSQL;
- просматривать историю операций и баланс;
- фильтровать, редактировать и удалять операции;
- анализировать доходы и расходы по категориям и периодам;
- устанавливать и контролировать бюджеты;
- импортировать банковские выписки;
- экспортировать данные в Excel;
- использовать шаблоны и напоминания;
- автоматически определять категории расходов;
- получать прогноз будущих расходов.

Цель разработки — создать удобную систему, которая объединяет быстрый ввод операций, визуальную аналитику, хранение данных и элементы машинного обучения.

1.6 Требования к системе

1.6.1 Функциональные требования

Система должна обеспечивать:

- регистрацию и идентификацию пользователя Telegram;
- добавление доходов и расходов;
- быстрый текстовый ввод операций;
- автоматическое предложение категории расхода;
- подтверждение или ручной выбор категории;
- просмотр баланса и истории операций;
- фильтрацию транзакций;
- редактирование и удаление операций;
- отображение аналитики и графиков;
- работу с бюджетами;
- работу с шаблонами операций;
- создание напоминаний;
- импорт банковских выписок;
- экспорт отчетов в Excel;
- прогнозирование расходов;
- административные функции.

1.6.2 Нефункциональные требования

К системе предъявляются следующие нефункциональные требования:

- удобный интерфейс Telegram-бота;
- наглядный интерфейс Mini App;
- быстрая обработка основных действий;
- модульная структура проекта;
- возможность дальнейшего расширения;
- централизованное хранение данных;
- корректная обработка ошибок ввода;
- стабильная работа основных пользовательских сценариев.

1.6.3 Требования к структуре системы

Система должна включать следующие компоненты:

- Telegram-бот;
- Telegram Mini App;
- серверную часть;
- REST API;
- базу данных PostgreSQL;
- модуль аналитики;
- модуль машинного обучения;
- модуль прогнозирования;
- модуль импорта выписок;
- модуль экспорта отчетов;
- модуль напоминаний;
- административные функции.

Такое разделение позволяет не смешивать пользовательский интерфейс, бизнес-логику, хранение данных и дополнительные функции.

1.6.4 Требования к программному обеспечению

Для разработки системы используются:

- Python [5];
- aiogram для Telegram-бота [4];
- Django для серверной части [7];
- Django REST Framework для REST API [8];
- PostgreSQL для базы данных [6];
- HTML, CSS и JavaScript для Mini App;
- scikit-learn для машинного обучения [10];
- Chart.js для построения интерактивных графиков в Mini App;

- Matplotlib для серверной генерации отдельных графиков и отчетных материалов [13];
- openpyxl и XlsxWriter для Excel-отчетов [14], [15];
- pdfplumber для обработки PDF-выписок [16];
- Postman для проверки REST API [17].

1.6.5 Требования к безопасности

Система должна обеспечивать базовый уровень защиты данных:

- хранение токенов и настроек вне исходного кода;
- разграничение данных по пользователям;
- ограничение доступа к административным функциям;
- проверку входных данных на сервере;
- защиту от изменения чужих данных;
- безопасную работу с базой данных;
- обработку ошибок без вывода служебной информации пользователю.

1.7 Выводы по разделу

В первом разделе была рассмотрена предметная область учета личных финансов, проанализированы существующие решения и выделены их основные ограничения. Было показано, что простые Telegram-боты удобны для быстрого ввода операций, но не всегда подходят для полноценной аналитики, бюджетирования и прогнозирования.

На основе анализа была сформулирована задача разработки системы @fffinansial_bot, которая объединяет Telegram-бот, Mini App, REST API, PostgreSQL, аналитику и модуль машинного обучения. Также были определены основные функциональные, нефункциональные, структурные, программные и защитные требования к системе.

2 Проектирование системы

2.1 Общая архитектура системы

Разрабатываемая система проектируется как клиент-серверное приложение. Такой подход позволяет разделить пользовательские интерфейсы, серверную логику, базу данных и дополнительные модули обработки информации.

В системе предусмотрены два пользовательских интерфейса: Telegram-бот и Telegram Mini App. Telegram-бот используется для быстрого добавления доходов и расходов, просмотра баланса, истории и выполнения основных действий. Mini App применяется для более удобной работы с транзакциями, аналитикой, бюджетами, категориями и прогнозами [2], [3].

Центральным компонентом является серверная часть. Она принимает запросы от пользовательских интерфейсов, проверяет данные, обращается к базе данных и передает информацию в модули аналитики, прогнозирования, импорта и экспорта. Для связи между клиентской и серверной частью используется REST API [8], [9].

Данные системы хранятся в PostgreSQL. Эта база данных подходит для хранения связанных сущностей: пользователей, категорий, транзакций, бюджетов, напоминаний и шаблонов операций [6], [19].



Рисунок 3 – Общая архитектура системы

На рисунке 3 показано, что пользователь работает с системой через Telegram-бота или Mini App, а основная обработка данных выполняется на серверной стороне.

2.2 Проектирование базы данных

База данных проектируется как центральное хранилище системы. В ней должны храниться все основные данные пользователя: профиль, категории, операции, бюджеты, напоминания, импортированные выписки и шаблоны операций.

Основными таблицами базы данных являются:

- users — хранит данные пользователей Telegram;
- categories — хранит пользовательские категории доходов и расходов;
- transactions — хранит финансовые операции;
- budgets — хранит лимиты расходов по категориям;
- reminders — хранит настройки напоминаний;
- budget_forecast — хранит прогнозные значения бюджета;
- statement_imports — хранит сведения об импортированных выписках;
- transaction_templates — хранит шаблоны повторяющихся операций.

Центральной таблицей является users, так как остальные данные связаны с конкретным пользователем. Таблица transactions связана с пользователем и категорией. Для поддержки автоматической категоризации в ней предусмотрены поля suggested_category_id и is_category_accepted. Они позволяют хранить предложенную моделью категорию и признак того, была ли она подтверждена пользователем.

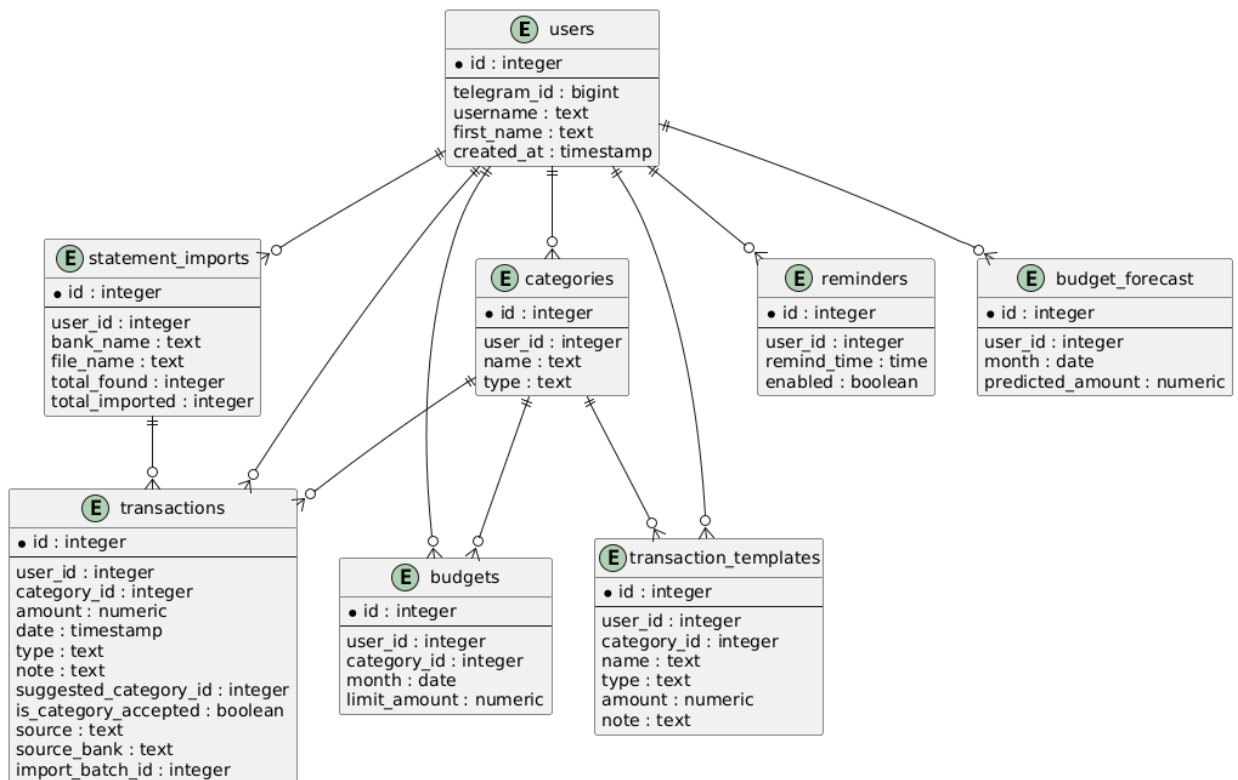


Рисунок 4 – ER-диаграмма базы данных

На рисунке 4 показаны основные таблицы базы данных и связи между ними. Такая структура позволяет хранить как базовые операции, так и дополнительные данные для аналитики, импорта, бюджетов и машинного обучения.

2.3 Проектирование Telegram-бота

Telegram-бот проектируется как основной инструмент быстрого взаимодействия с системой. Через него пользователь может добавить доход или расход, посмотреть баланс, открыть историю, выполнить импорт выписки, экспортировать отчет и настроить напоминания.

Взаимодействие строится через команды, кнопки и пошаговые сценарии. Для сложных действий используется FSM, то есть конечный автомат. Он позволяет боту понимать, на каком этапе находится пользователь:

вводит сумму, выбирает категорию, указывает дату или подтверждает операцию.

Основные функции Telegram-бота:

- старт и регистрация пользователя;
- главное меню;
- добавление дохода;
- добавление расхода;
- быстрый текстовый ввод;
- подтверждение категории, предложенной ML-модулем;
- просмотр баланса;
- просмотр истории;
- импорт банковской выписки;
- экспорт отчета;
- напоминания;
- профиль;
- административные функции.



Рисунок 5 – Структура функций Telegram-бота

На рисунке 5 показаны основные функции Telegram-бота. Бот используется для быстрых действий, которые пользователь чаще всего выполняет в повседневной работе.

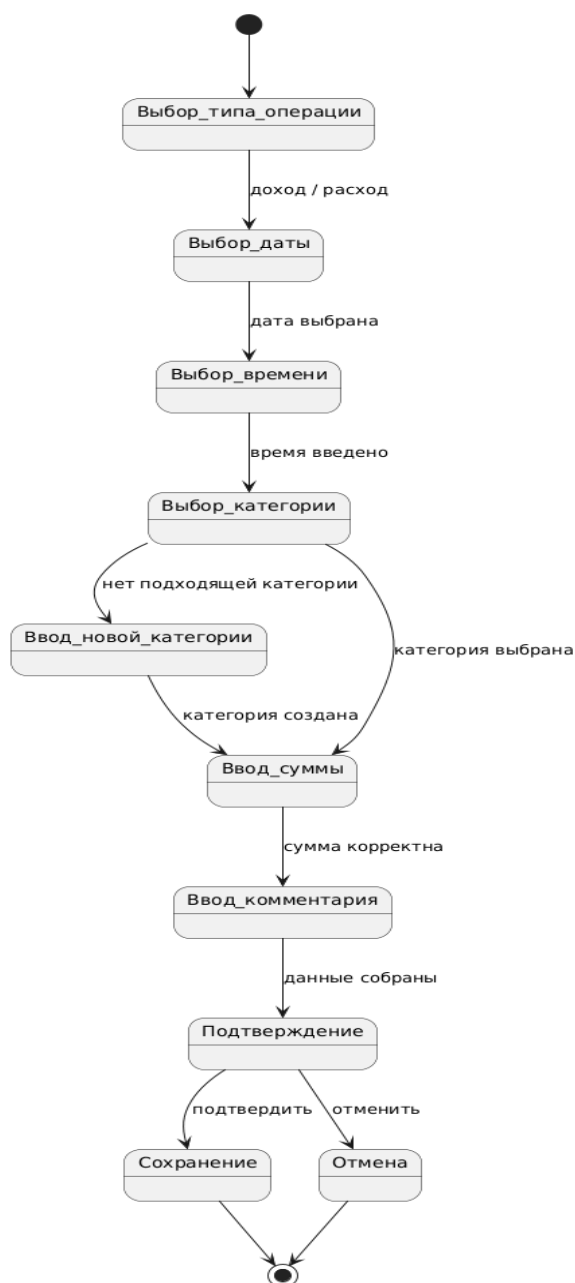


Рисунок 6 – Диаграмма состояний Telegram-бота

На рисунке 6 показан общий сценарий работы Telegram-бота при добавлении операции. Отдельно выделено подтверждение категории, предложенной модулем машинного обучения.

2.4 Проектирование Mini App

Mini App проектируется как расширенный графический интерфейс системы. Он нужен для тех задач, которые неудобно выполнять через чат: просмотр графиков, фильтрация операций, управление бюджетами, категориями, шаблонами и прогнозами.

Mini App не должен содержать основную бизнес-логику. Он отправляет запросы к REST API и отображает полученные данные. Такой подход позволяет не дублировать правила обработки данных в разных интерфейсах.

Основные разделы Mini App:

- главная страница;
- список транзакций;
- добавление и редактирование операции;
- аналитика;
- бюджеты;
- прогноз расходов;
- категории;
- шаблоны операций;
- профиль пользователя.



Рисунок 7 – Структура разделов Mini App

На рисунке 7 показана структура Mini App. Интерфейс дополняет Telegram-бот и используется для более подробной работы с финансовыми данными.

2.5 Проектирование REST API

REST API проектируется как единый слой связи между пользовательскими интерфейсами и серверной частью. Через него Telegram-бот и Mini App получают данные, создают операции, запрашивают аналитику и прогноз.

В системе предусмотрены следующие группы API:

- авторизация Mini App;
- данные текущего пользователя;
- категории;
- шаблоны операций;
- транзакции;
- быстрый ввод операций;
- сводка по финансам;
- аналитика;
- прогноз расходов;
- бюджеты.

Основные маршруты API:

- /api/health/;
- /api/miniapp/auth/;
- /api/me/;
- /api/categories/;
- /api/templates/;
- /api/transactions/;
- /api/transactions/quick-add/preview/;

- /api/transactions/quick-add/create/;
- /api/summary/;
- /api/analytics/overview/;
- /api/analytics/series/;
- /api/forecast/;
- /api/insights/;
- /api/budgets/;
- /api/budgets/summary/.



Рисунок 8 – Структура REST API системы

На рисунке 8 показаны основные группы REST API. Такое разделение упрощает сопровождение системы и позволяет расширять функциональность без изменения всей архитектуры.

Для защиты пользовательских данных REST API должен проверять принадлежность каждого запроса конкретному пользователю. После авторизации Mini App пользователь получает API-ключ, который передается в последующих запросах. На стороне сервера по этому ключу определяется пользователь, после чего все операции с категориями, транзакциями,

бюджетами, шаблонами и аналитикой выполняются только в пределах его данных.

Такой подход позволяет не передавать идентификатор пользователя вручную в каждом запросе и снижает риск получения чужих данных. Если API-ключ отсутствует или указан неверно, сервер должен вернуть ошибку доступа. Если пользователь пытается обратиться к несуществующей или чужой записи, сервер не должен возвращать эти данные.

2.6 Проектирование аналитики

Модуль аналитики нужен для обработки накопленных транзакций и формирования понятной финансовой сводки. Он должен рассчитывать не только отдельные операции, но и итоговые показатели за выбранный период.

В модуле аналитики проектируются следующие расчеты:

- сумма доходов;
- сумма расходов;
- текущий баланс;
- количество операций;
- расходы по категориям;
- динамика расходов по дням;
- динамика по месяцам;
- сравнение текущего и предыдущего периода;
- средний расход;
- самая затратная категория;
- самый затратный день.

Результаты аналитики используются в Mini App и в отдельных функциях Telegram-бота. Для визуального представления данных применяются графики [13].



Рисунок 9 – Схема аналитической обработки данных

На рисунке 9 показано, как данные из таблицы транзакций используются для расчета финансовых показателей и построения аналитики.

2.7 Проектирование модуля машинного обучения

Модуль машинного обучения проектируется для автоматической категоризации расходов и формирования прогноза будущих трат. Его задача заключается не в полной замене действий пользователя, а в сокращении количества ручных операций при ведении учета. Пользователь по-прежнему может самостоятельно выбрать категорию, однако система должна предложить наиболее подходящий вариант на основе ранее сохраненных данных.

При добавлении расхода в качестве входных данных используются описание операции, сумма и при необходимости дополнительные сведения о выбранной пользователем категории. Описание операции является основным текстовым признаком, так как в нем обычно содержатся слова, указывающие

на смысл расхода: «такси», «продукты», «подписка», «одежда», «кафе» и другие. Сумма используется как дополнительный признак, поскольку для некоторых категорий характерны определенные диапазоны значений.

Процесс автоматической категоризации состоит из нескольких этапов. Сначала выполняется предварительная обработка пользовательского ввода: текст приводится к единому виду, удаляются лишние символы, после чего проверяется наличие ключевых слов. Такой этап нужен для простых и очевидных случаев, когда категорию можно определить без применения модели машинного обучения. Например, слова «такси» или «бензин» могут указывать на транспорт, а слово «продукты» — на соответствующую категорию расходов.

Если категория не была определена с помощью правил, используется модель машинного обучения. Для преобразования текстового описания в числовое представление применяется TF-IDF-векторизация [11]. Такой способ позволяет учитывать важность слов в коротких пользовательских описаниях. Для классификации расходов применяется логистическая регрессия, так как она подходит для задач многоклассовой классификации и может использоваться при ограниченном объеме обучающих данных [12]. Работа с моделью выполняется средствами библиотеки `scikit-learn` [10].

Результат работы ML-модуля не должен автоматически сохраняться без участия пользователя. Система предлагает предполагаемую категорию, после чего пользователь подтверждает ее или выбирает другую. Такой подход снижает риск ошибочной категоризации и одновременно позволяет использовать подтвержденные операции как основу для дальнейшего улучшения качества модели.

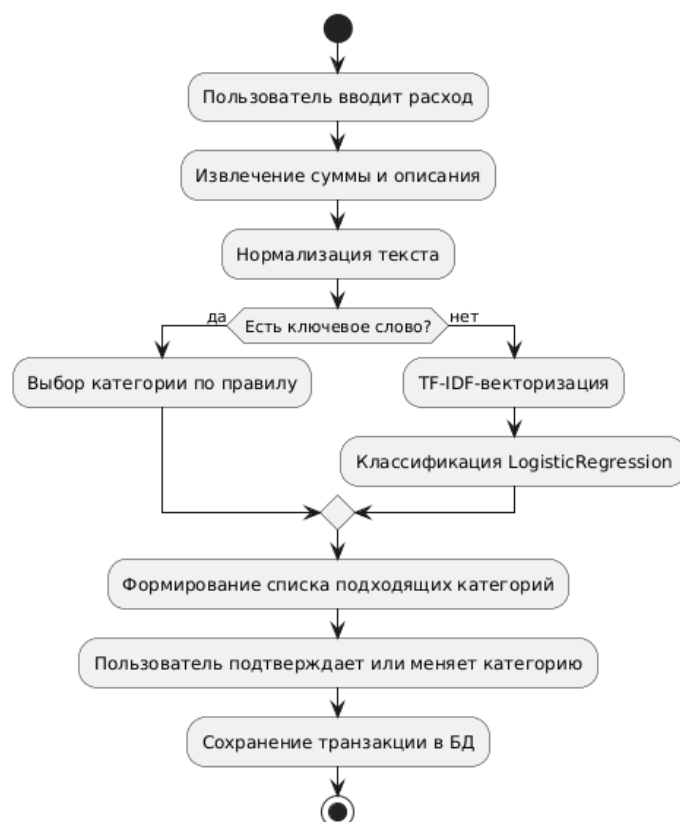


Рисунок 10 – Алгоритм автоматической категоризации расхода

На рисунке 10 представлена схема работы модуля автоматической категоризации. Сначала данные операции передаются на предварительную обработку, затем выполняется проверка по ключевым словам, а при отсутствии точного совпадения используется ML-модель. Итоговая категория передается пользователю для подтверждения.

Второй задачей ML-модуля является прогнозирование расходов. Для этого используются ранее сохраненные транзакции пользователя. Система анализирует расходы за прошлые периоды, выделяет динамику по месяцам и рассчитывает предполагаемое значение на следующий период. Если накопленных данных недостаточно, прогноз строится по более простому принципу — на основе последних доступных значений.



Рисунок 11 – Схема работы модуля прогнозирования расходов

На рисунке 11 показано, что прогнозирование связано с историей расходов пользователя и аналитическим модулем. Полученный результат отображается в Mini App и помогает пользователю заранее оценить возможные траты. При этом прогноз рассматривается как вспомогательный показатель, а не как точное финансовое предсказание.

2.8 Проектирование дополнительных модулей

Помимо основных функций учета, в системе предусматриваются дополнительные модули, которые закрывают практические сценарии работы с финансовыми данными: перенос операций из банковских документов, выгрузку отчетов, использование повторяющихся операций, напоминания и административное сопровождение.

Модуль импорта выписок предназначен для загрузки банковских документов и переноса операций в общую историю пользователя. При проектировании данного модуля учитывается, что данные из выписки не должны сразу сохраняться в базу без проверки. Поэтому после загрузки файла система извлекает операции, приводит даты и суммы к единому формату, показывает предварительный просмотр и только после подтверждения

добавляет выбранные записи в базу данных. Для обработки PDF-выписок используется библиотека pdfplumber [16].

Модуль экспорта отчетов нужен для выгрузки финансовых данных во внешний файл Excel. В отчет должны попадать основные сведения об операциях: дата, время, тип операции, категория, сумма и комментарий. Такой формат удобен для хранения копии данных, передачи отчета и последующей обработки вне системы. Для формирования Excel-файлов используются библиотеки openpyxl и XlsxWriter [14], [15].

Модуль напоминаний предназначен для случаев, когда пользователь забывает регулярно вносить операции или проверять расходы. При проектировании данного модуля предусматривается отправка уведомлений через Telegram. Это позволяет оставить взаимодействие с пользователем в той же среде, где выполняется быстрый ввод доходов и расходов.

Модуль шаблонов операций используется для повторяющихся платежей и доходов. К таким операциям можно отнести оплату связи, подписки, коммунальные платежи или регулярные поступления. Пользователь может сохранить типовую операцию и затем создавать новую запись на ее основе без повторного ввода всех данных.

Административные функции проектируются для сопровождения системы. Они позволяют просматривать пользователей, контролировать корректность данных и выполнять служебные действия при эксплуатации приложения. Эти возможности не относятся к основному пользовательскому сценарию, но необходимы для поддержки работы системы.

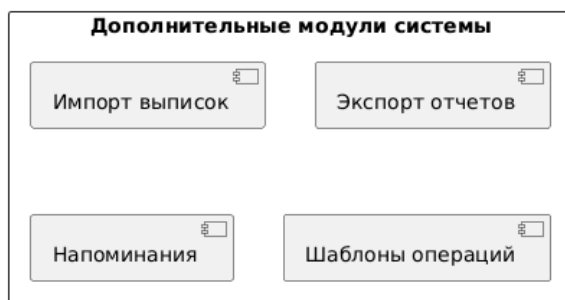


Рисунок 12 – Структура дополнительных модулей системы

На рисунке 12 показаны дополнительные модули, которые дополняют базовый учет доходов и расходов. Их наличие делает систему более пригодной для повседневного использования, так как пользователь может не только вручную добавлять операции, но и импортировать данные, выгружать отчеты, использовать шаблоны и получать напоминания.

2.9 Выводы по разделу

Во втором разделе была спроектирована структура системы учета, анализа и прогнозирования личных финансов. Были определены основные компоненты: Telegram-бот, Mini App, серверная часть, REST API, база данных PostgreSQL, модуль аналитики, модуль машинного обучения и дополнительные сервисные модули.

Также была спроектирована база данных, описаны основные пользовательские сценарии, структура REST API, логика аналитики, алгоритм автоматической категоризации расходов и подход к прогнозированию. Полученные проектные решения используются далее при реализации программного обеспечения.

3 Реализация программного обеспечения

3.1 Выбор средств разработки

Для реализации системы был выбран язык Python, так как он подходит для разработки серверной логики, Telegram-ботов, работы с базами данных, аналитики и машинного обучения [5]. Telegram-бот реализован с использованием фреймворка aiogram, который поддерживает асинхронную обработку сообщений, команд и callback-запросов [4].

Серверная часть и REST API реализованы на Django и Django REST Framework [7], [8]. Для хранения данных используется PostgreSQL [6]. Mini App реализован как веб-интерфейс с использованием HTML, CSS и JavaScript. Для машинного обучения применяется scikit-learn [10], для построения интерактивных графиков в Mini App используется Chart.js, а Matplotlib может применяться для серверной генерации отдельных графиков и отчетных материалов [13], для Excel-выгрузок — openpyxl и XlsxWriter [14], [15], для обработки PDF-выписок — pdfplumber [16].

Такой набор средств позволил реализовать систему как единый программный продукт, включающий Telegram-бот, Mini App, серверную часть, REST API, базу данных, аналитику и ML-модули.

3.2 Реализация базы данных

База данных реализована в PostgreSQL. Она используется для хранения пользователей, категорий, транзакций, бюджетов, напоминаний, шаблонов операций и данных, связанных с импортом выписок.

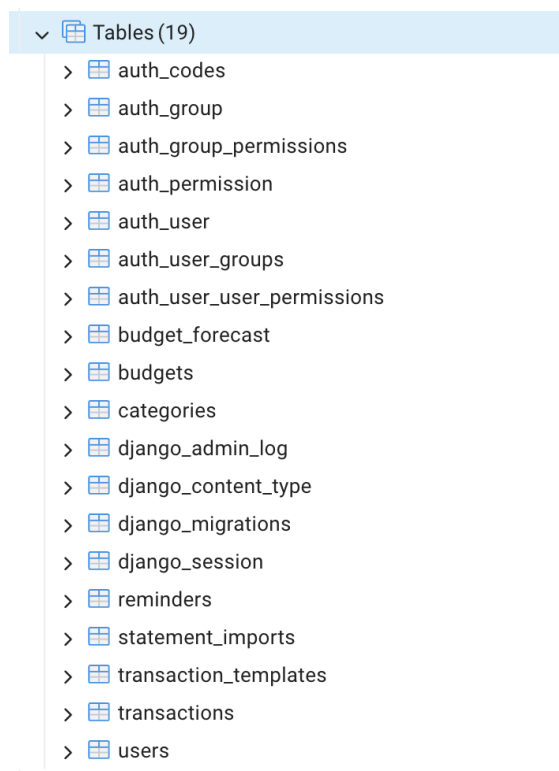


Рисунок 13 – Таблицы базы данных в pgAdmin

На рисунке 13 показаны таблицы базы данных в pgAdmin. Видно, что данные системы разделены по отдельным сущностям, что упрощает хранение и дальнейшую обработку информации.

Структура базы данных создается с помощью SQL-скрипта. В нем определяются таблицы, поля, типы данных и связи между сущностями.

```

CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    telegram_id BIGINT UNIQUE NOT NULL,
    username TEXT,
    first_name TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- =====
-- Категории доходов и расходов
-- =====

CREATE TABLE IF NOT EXISTS categories (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    type TEXT CHECK (type IN ('income', 'expense')) NOT NULL
);

-- =====
-- Транзакции
-- =====

CREATE TABLE IF NOT EXISTS transactions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    category_id INTEGER REFERENCES categories(id) ON DELETE SET NULL,
    amount NUMERIC(10,2) NOT NULL,
    date TIMESTAMP NOT NULL,
    type TEXT CHECK (type IN ('income', 'expense')) NOT NULL,
    note TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

```

Рисунок 14 – Фрагмент SQL-скрипта создания базы данных

На рисунке 14 показан фрагмент SQL-скрипта. Он используется для начальной подготовки базы данных перед запуском системы.

3.3 Структура программного проекта

Проект имеет модульную структуру. Основные функции Telegram-бота вынесены в папку handlers, работа с базой данных — в database, вспомогательные функции — в services, состояния FSM — в states, а серверная часть Mini App и REST API расположена в папке web.

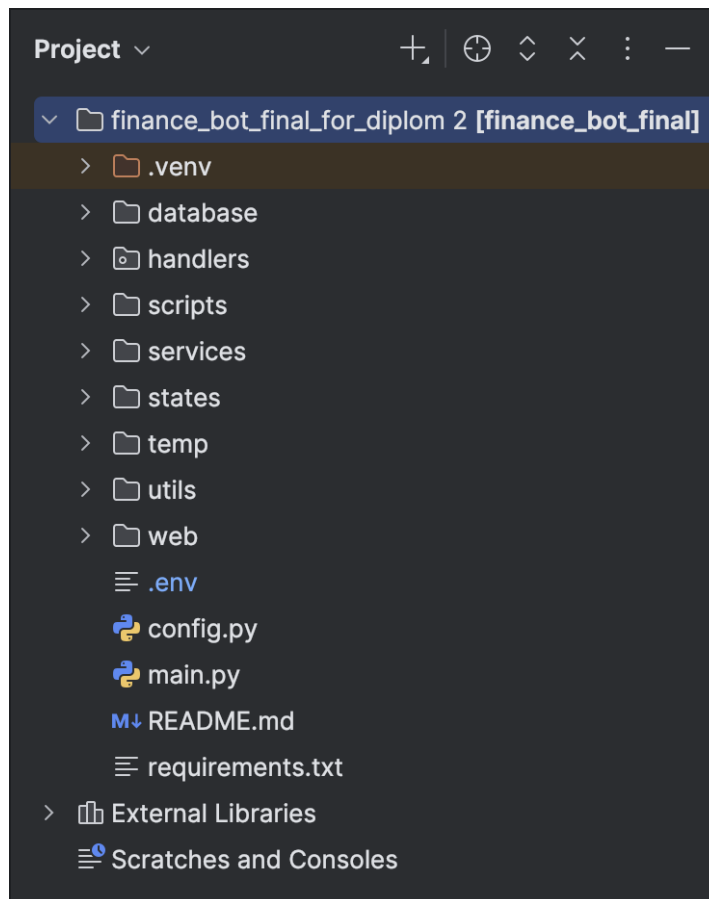


Рисунок 15 – Структура программного проекта

На рисунке 15 показана структура проекта. Такое разделение позволяет не смешивать обработчики Telegram-бота, серверную часть, работу с базой данных и дополнительные сервисные модули.

Основные файлы проекта:

- main.py — запуск Telegram-бота;
- config.py — подключение настроек;
- database/db.py — подключение к PostgreSQL;
- database/repository.py — функции работы с данными;
- handlers/start.py — запуск и главное меню;
- handlers/quick_add.py — быстрый текстовый ввод;
- handlers/balance.py — просмотр баланса;
- handlers/history.py — история операций;
- handlers/export.py — экспорт в Excel;

- `handlers/import_statement.py` — импорт банковских выписок;
- `handlers/reminders.py` — напоминания;
- `handlers/admin.py` — административные функции;
- `web/finance/api.py` — REST API;
- `web/finance/models.py` — модели серверной части;
- `web/finance/urls.py` — маршруты API.

3.4 Реализация Telegram-бота

Telegram-бот реализован как основной интерфейс быстрого взаимодействия с пользователем. Он позволяет добавлять операции, просматривать баланс, получать последние транзакции, импортировать выписки, экспортировать отчеты и открывать Mini App.

Работа пользователя начинается с команды `/start`. При первом запуске бот проверяет наличие пользователя в базе данных. Если запись отсутствует, создается новый пользователь с Telegram ID, username и именем.

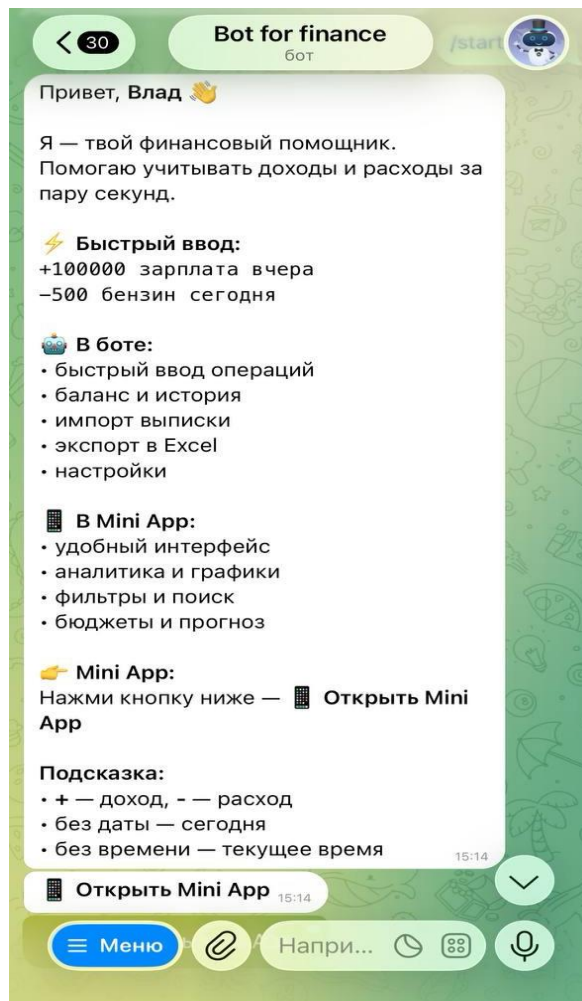


Рисунок 16 – Стартовый экран Telegram-бота

На рисунке 16 показан стартовый экран бота. Пользователь получает краткое описание возможностей системы, примеры быстрого ввода и кнопку для открытия Mini App.

Главное меню реализовано через клавиатуру Telegram. В него вынесены основные действия: баланс, последние операции, импорт выписки, экспорт в Excel, настройки и переход в Mini App.

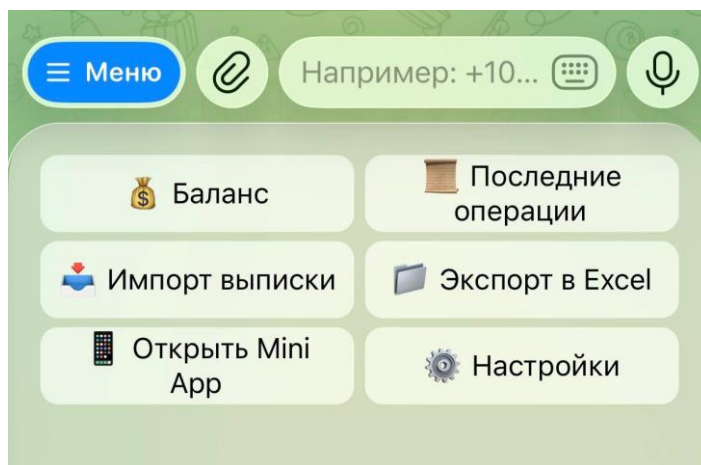


Рисунок 17 – Главное меню Telegram-бота

На рисунке 17 показано главное меню Telegram-бота. Благодаря кнопкам пользователь может выполнять основные действия без ручного ввода команд.

Одной из основных функций является быстрый текстовый ввод. Пользователь может отправить строку вида +100000 зарплата вчера 21:21 или -500 бензин сегодня. После этого система определяет тип операции, сумму, описание, дату и время.

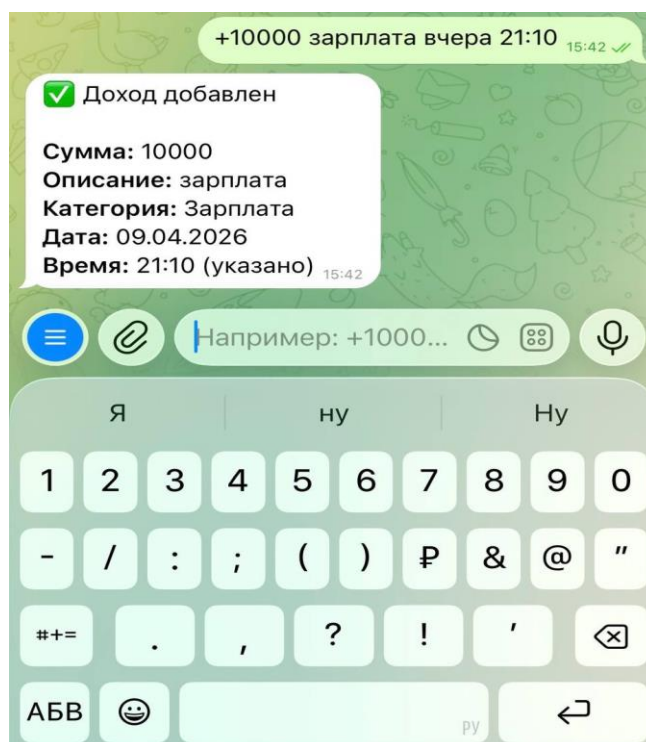


Рисунок 18 – Быстрое добавление дохода

На рисунке 18 показан пример быстрого добавления дохода. Бот распознает введенную строку, определяет тип операции и сохраняет запись в базу данных.

Для расходов дополнительно используется модуль автоматического определения категории. После разбора строки бот передает описание и сумму в ML-модуль, получает предполагаемую категорию и предлагает пользователю подтвердить или изменить результат.

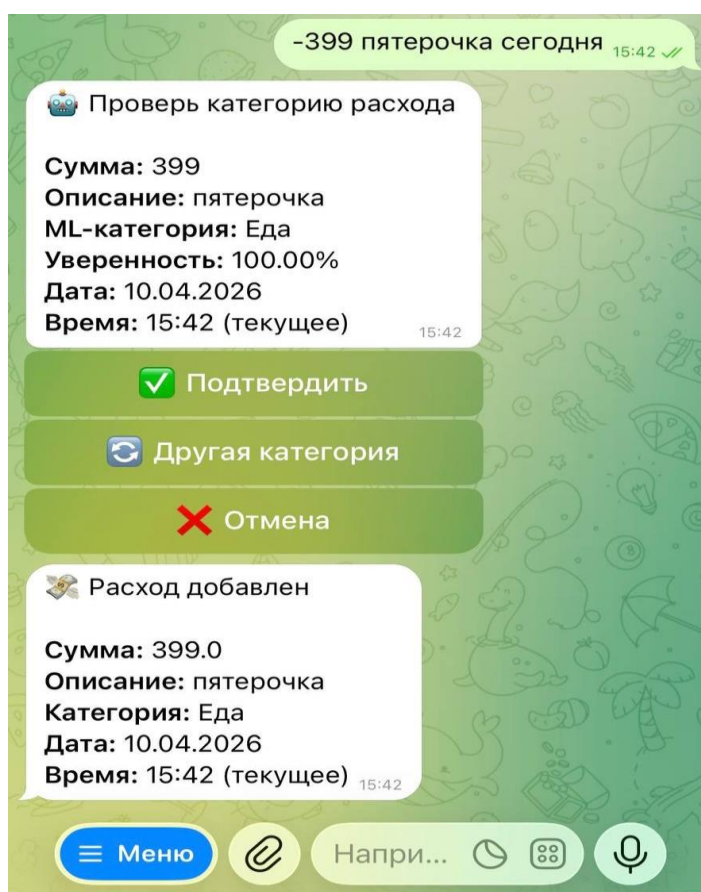


Рисунок 19 – Быстрое добавление расхода с автоматическим определением категории

На рисунке 19 показан пример добавления расхода. Бот определяет предполагаемую категорию, показывает уровень уверенности и предлагает подтвердить результат.

Для быстрого просмотра финансового состояния реализован обработчик баланса. Он рассчитывает доходы, расходы и итоговый баланс за всё время и за текущий месяц.

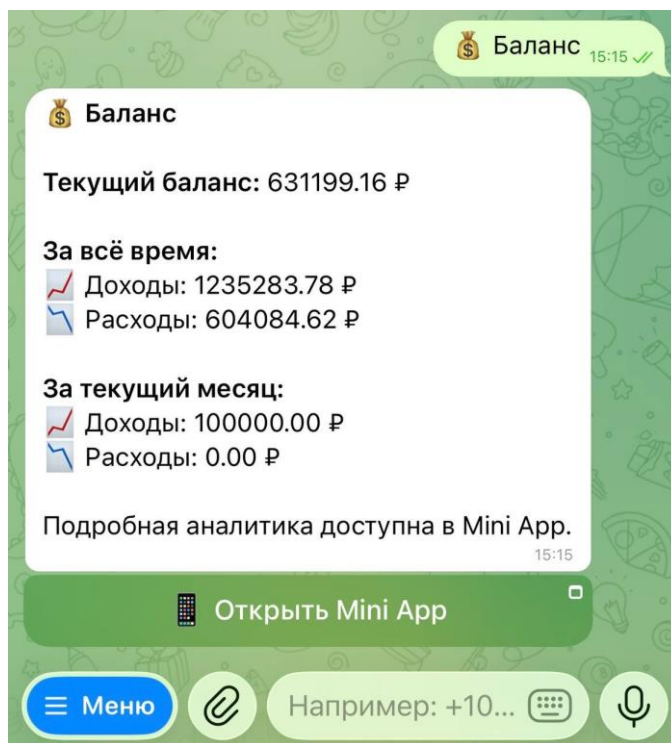


Рисунок 20 – Просмотр баланса в Telegram-боте

На рисунке 20 показан вывод текущего баланса. Такой экран удобен для быстрого контроля состояния личных финансов.

История операций выводит последние транзакции пользователя с указанием суммы, категории, даты и комментария.

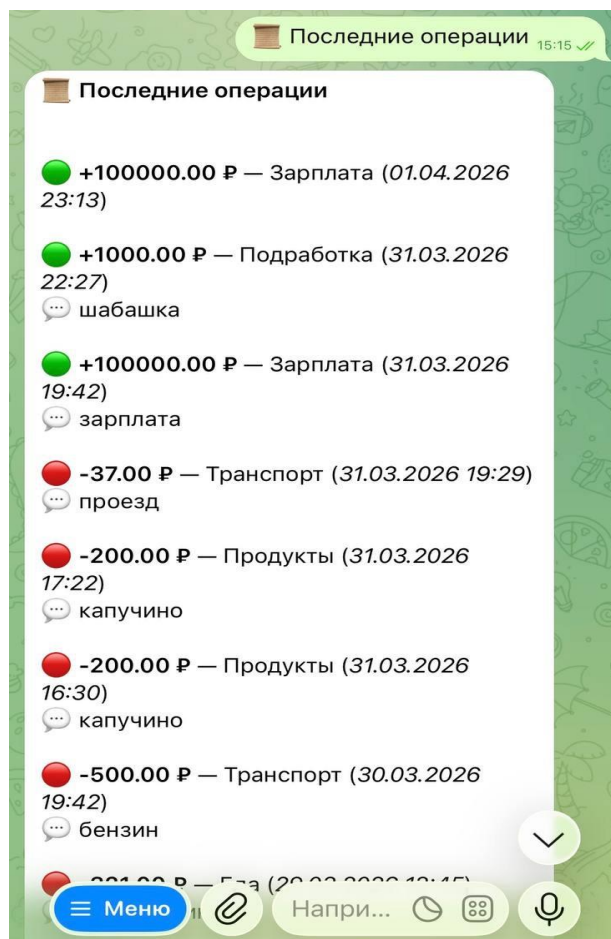


Рисунок 21 – Просмотр последних операций

На рисунке 21 показан список последних операций. Бот выводит краткую историю без необходимости открывать Mini App.

Для выгрузки данных реализован экспорт в Excel. Пользователь выбирает, нужно выгрузить все операции или данные за конкретный месяц.

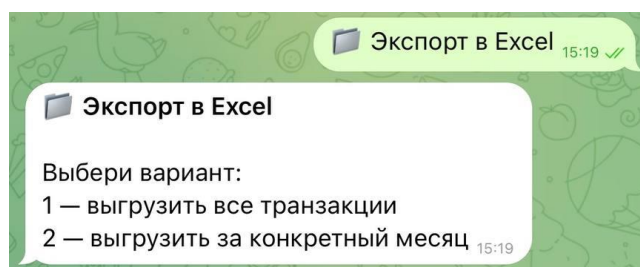


Рисунок 22 – Выбор параметров экспорта в Excel

На рисунке 22 показан начальный этап экспорта. Пользователь выбирает нужный вариант выгрузки.

После выбора параметров бот формирует Excel-файл и отправляет его пользователю в чат.

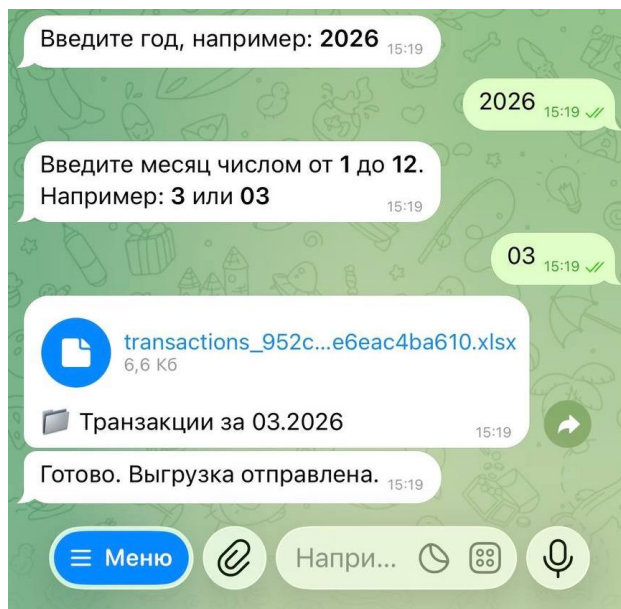


Рисунок 23 – Результат экспорта данных в Excel

На рисунке 23 показан результат экспорта. Пользователь получает готовый файл с финансовыми операциями.

Для импорта банковских выписок реализован отдельный сценарий. Сначала пользователь выбирает банк, затем отправляет PDF-файл выписки.

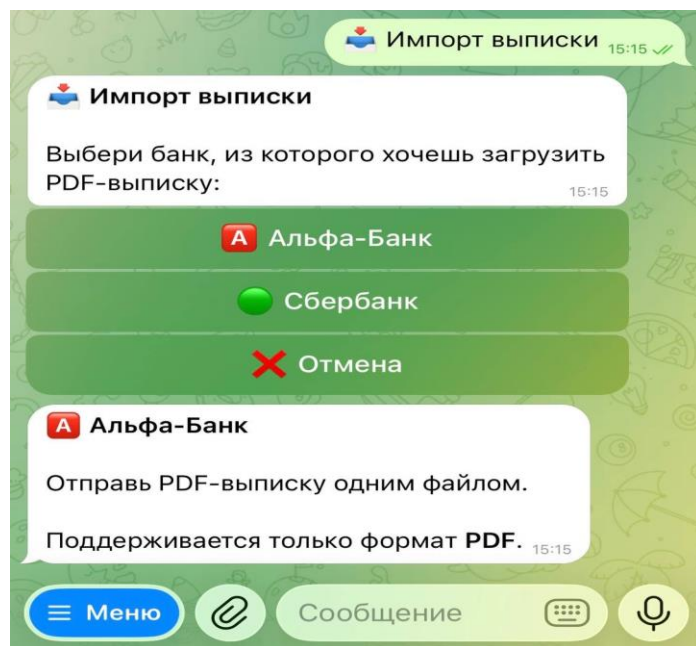


Рисунок 24 – Выбор банка для импорта выписки

На рисунке 24 показан выбор банка для импорта. После этого бот ожидает загрузку PDF-файла.

Перед сохранением операций бот показывает предварительный просмотр найденных данных.

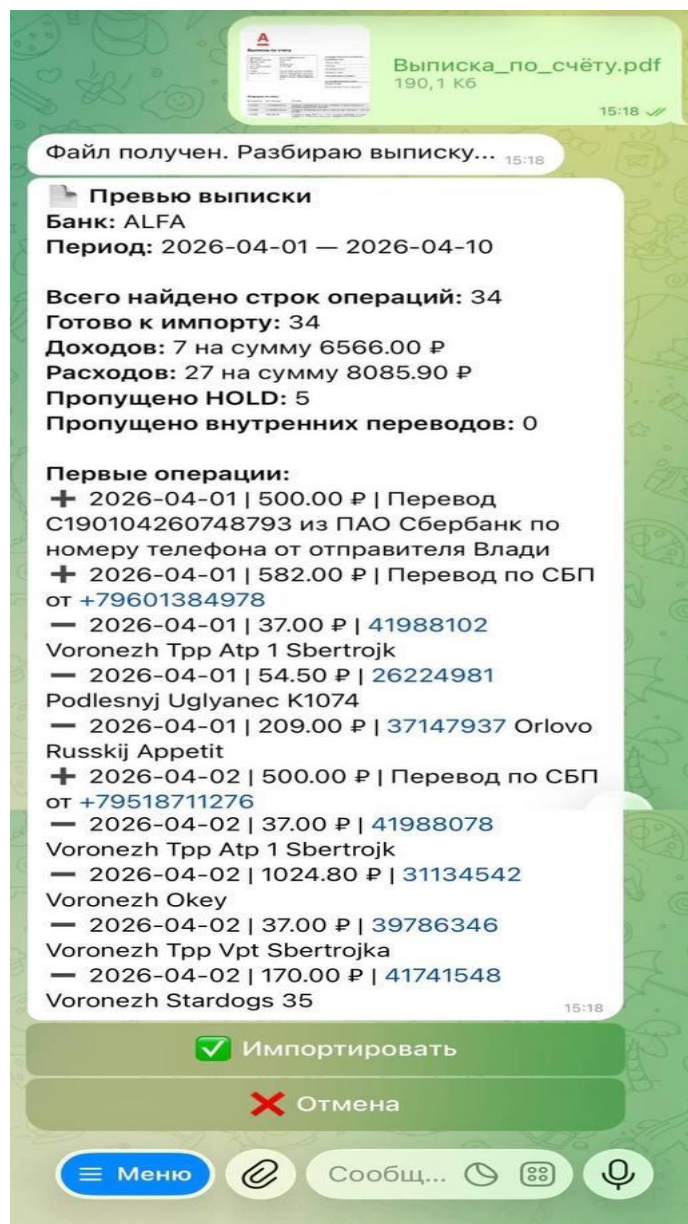


Рисунок 25 – Предварительный просмотр данных выписки

На рисунке 25 показан предварительный просмотр выписки. Пользователь может проверить найденные операции перед импортом.

В боте также реализован раздел настроек, из которого пользователь может открыть профиль и управление напоминаниями.

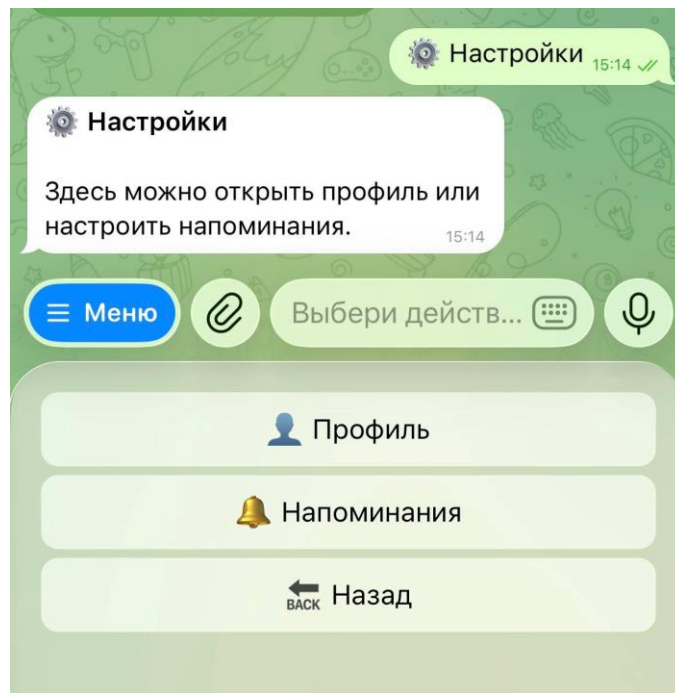


Рисунок 26 – Раздел настроек Telegram-бота

На рисунке 26 показан раздел настроек. Через него пользователь переходит к профилю и напоминаниям.

Профиль формируется на основе данных пользователя и его финансовой статистики.

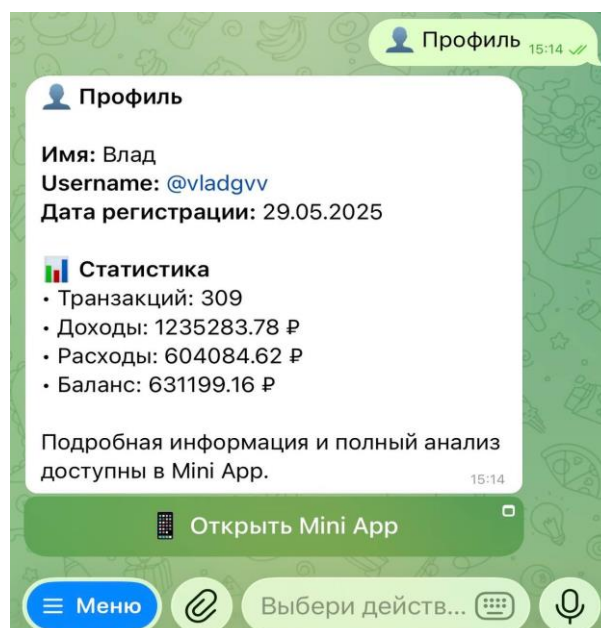


Рисунок 27 – Профиль пользователя в Telegram-боте

На рисунке 27 показан профиль пользователя. В нем отображаются регистрационные данные и краткая статистика по операциям.

Напоминания реализованы через отдельный сценарий с кнопками. Пользователь может включить или отключить уведомления.

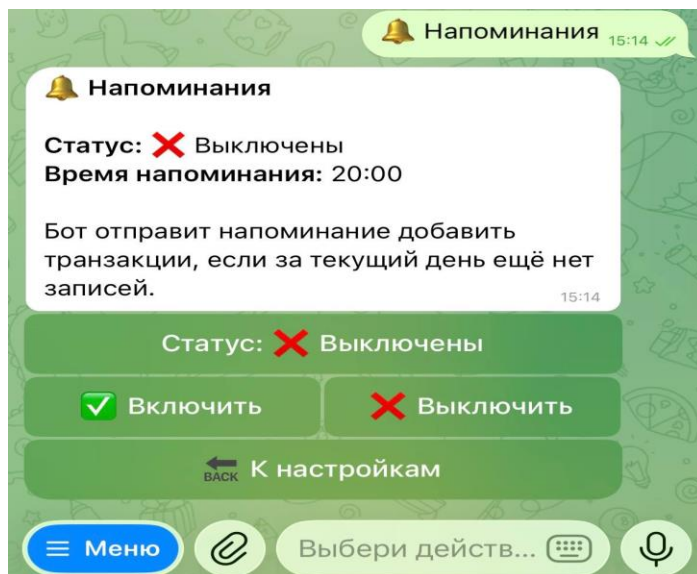


Рисунок 28 – Настройка напоминаний

На рисунке 28 показан интерфейс настройки напоминаний. Эта функция помогает пользователю не забывать вести учет.

Дополнительно в Telegram-боте реализованы административные функции для сопровождения системы.

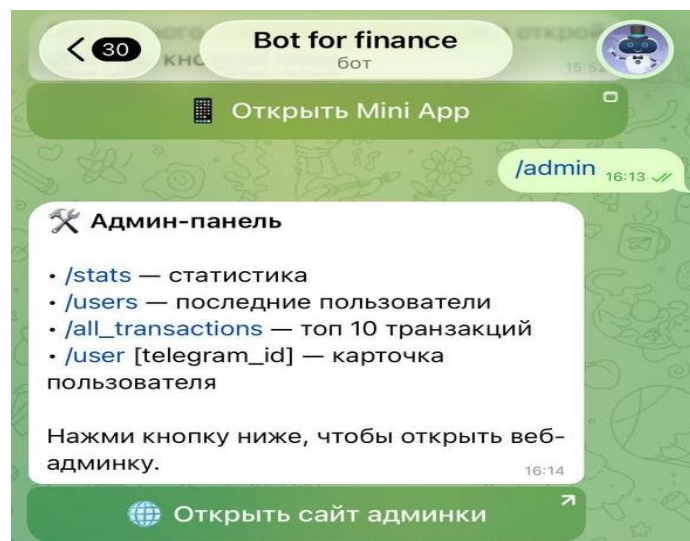


Рисунок 29 – Административные функции Telegram-бота

На рисунке 29 показан административный сценарий. Он используется для просмотра служебной информации и управления данными пользователей.

Таким образом, Telegram-бот реализован как быстрый интерфейс системы. Он закрывает ежедневные сценарии пользователя: ввод операций, просмотр баланса, история, импорт, экспорт, настройки и переход в Mini App.

3.5 Реализация Mini App

Mini App реализован как расширенный графический интерфейс системы. Он открывается внутри Telegram и используется для тех действий, которые неудобно выполнять через чат: просмотр графиков, управление транзакциями, бюджетами, категориями и прогнозами [3].

Авторизация Mini App выполняется через данные Telegram Web App. После открытия Mini App сервер получает данные пользователя, проверяет их и связывает с записью в базе данных. После успешной авторизации пользователю выдается API-ключ, который используется при дальнейших запросах к REST API. Это позволяет Mini App получать только данные текущего пользователя.

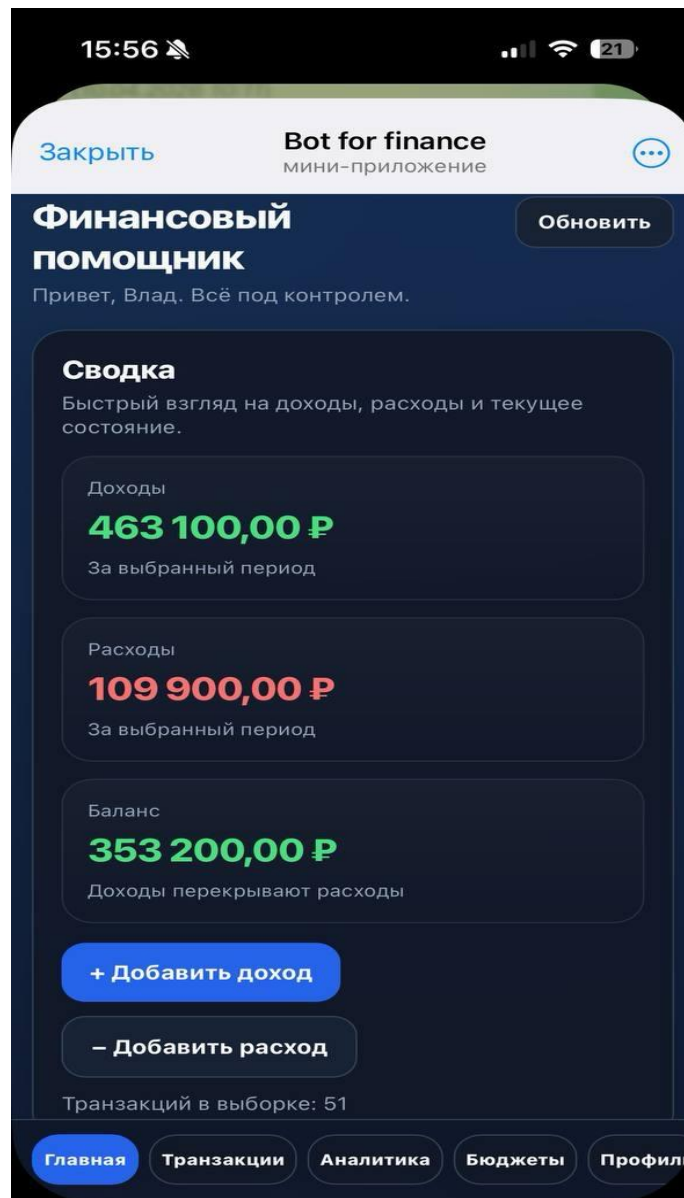


Рисунок 30 – Главный экран Mini App

На рисунке 30 показан главный экран Mini App. Пользователь видит доходы, расходы, баланс и кнопки быстрого добавления операций.

На главном экране также реализован блок умных карточек месяца. Он содержит дополнительные показатели: средний расход в день, безопасный лимит, остаток дней и статус бюджета.



Рисунок 31 – Умные карточки месяца

На рисунке 31 показаны умные карточки месяца. Они помогают пользователю быстрее оценить текущее финансовое состояние.

Раздел транзакций содержит список последних операций. В отличие от Telegram-бота, здесь пользователь видит операции в более удобном карточном виде.

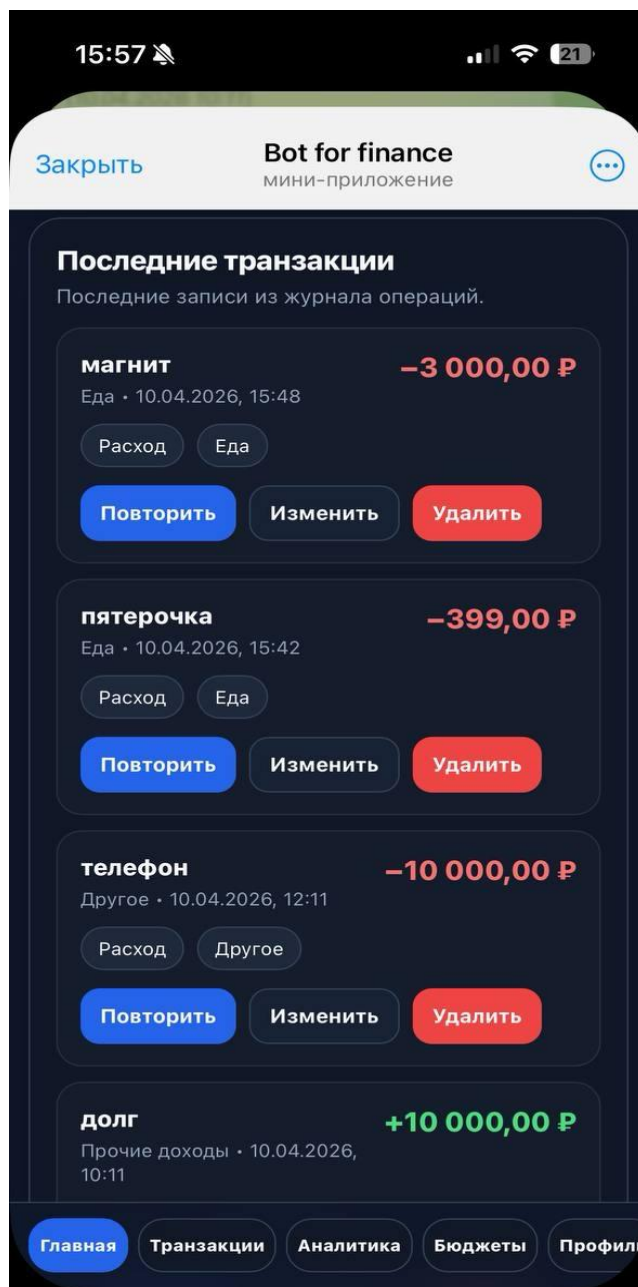


Рисунок 32 – Последние транзакции в Mini App

На рисунке 32 показаны последние транзакции. Каждая запись содержит сумму, категорию, дату и доступные действия.

Для более точной работы с историей реализован отдельный экран управления транзакциями. Он поддерживает фильтрацию по типу, категории, датам и тексту.

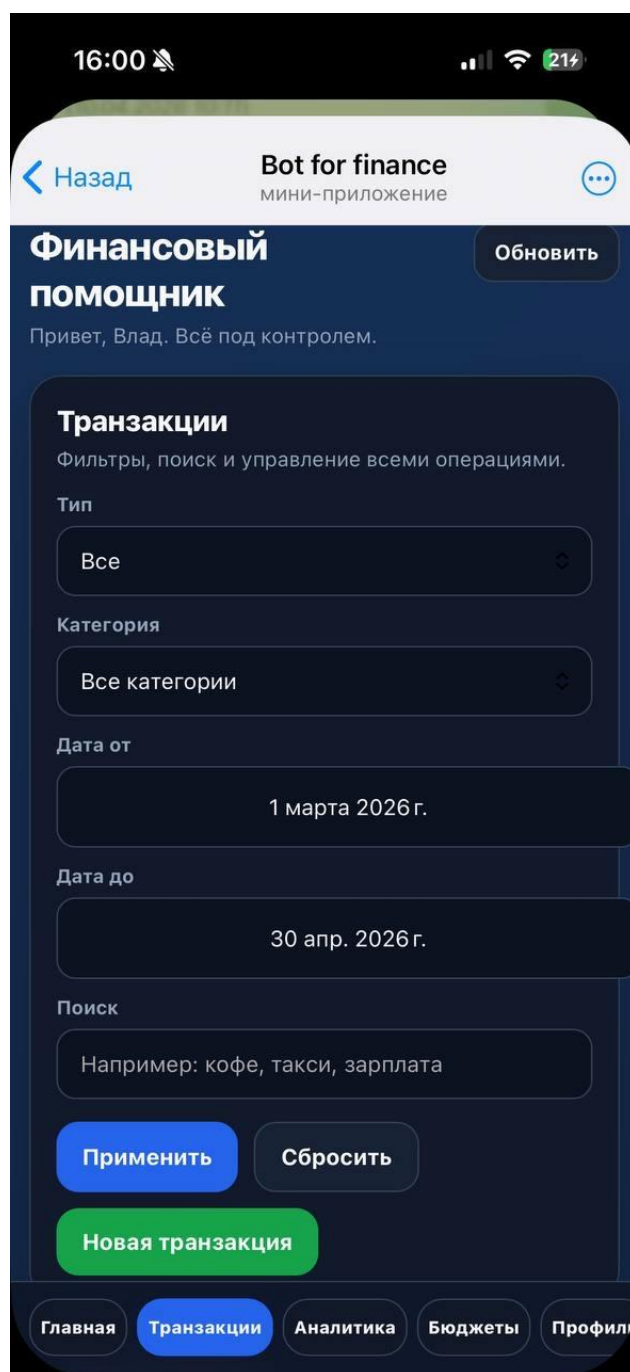


Рисунок 33 – Экран управления транзакциями

На рисунке 33 показан раздел транзакций. Фильтры помогают быстро найти нужную операцию.

Добавление новой операции через Mini App выполняется через отдельную форму.

The screenshot displays a mobile application interface for 'Bot for finance'. At the top, the status bar shows the time 15:59, signal strength, Wi-Fi, and battery level at 21%. The app header includes a 'Заккрыть' (Close) button, the title 'Bot for finance' with the subtitle 'мини-приложение' (mini-application), and a menu icon. Below the header, the balance is shown as 'Баланс 353 200,00 Р' with the note 'Доходы перекрывают расходы' (Income covers expenses). A modal form titled 'Новая транзакция' (New Transaction) is open, featuring a close button (X). The form contains the following fields: 'Тип' (Type) set to 'Расход' (Expense), 'Сумма' (Sum) set to '449', 'Категория' (Category) set to 'Подписки' (Subscriptions), 'Дата и время' (Date and time) set to '10 апр. 2026 г., 15:58', and 'Комментарий' (Comment) set to 'Яндекс плюс'. At the bottom of the form are two buttons: 'Сохранить' (Save) and 'Отмена' (Cancel).

Рисунок 34 – Форма добавления новой транзакции

На рисунке 34 показана форма создания операции. Пользователь указывает тип, сумму, категорию, дату и комментарий.

В Mini App также реализован быстрый ввод одной строкой, аналогичный быстрому вводу в Telegram-боте.

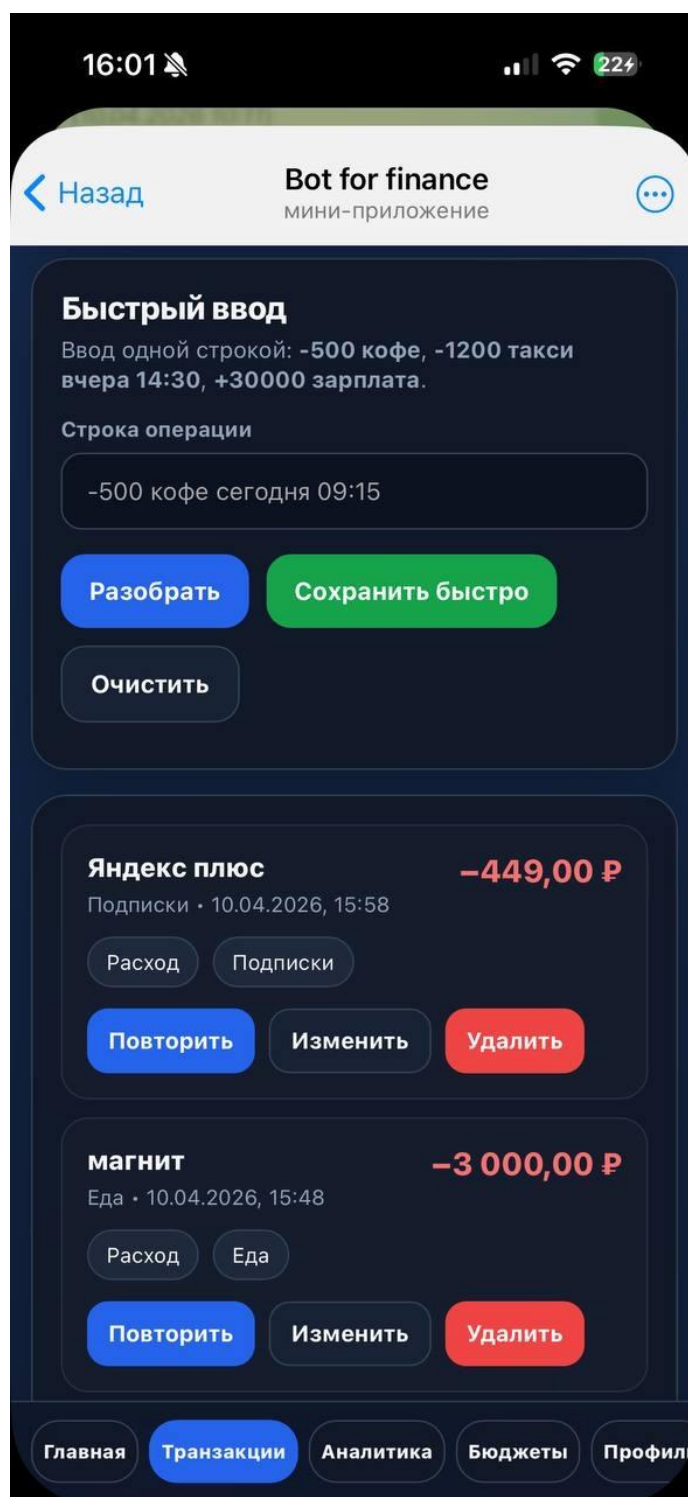


Рисунок 35 – Быстрый ввод операций в Mini App

На рисунке 35 показан быстрый ввод операции в Mini App. Система разбирает строку и сохраняет операцию в упрощенном формате.

Раздел аналитики показывает структуру расходов и сравнение с предыдущим периодом.



Рисунок 36 – Аналитика по структуре расходов

На рисунке 36 показана аналитика расходов по категориям. Такой экран позволяет увидеть основные направления трат.

Для анализа расходов по дням реализован отдельный графический блок.



Рисунок 37 – Анализ расходов по дням

На рисунке 37 показаны расходы по дням. Этот график помогает определить дни с наибольшими тратами.

Дополнительно реализовано сравнение текущего периода с предыдущим.



Рисунок 38 – Сравнение текущего периода с предыдущим

На рисунке 38 показано сравнение доходов, расходов и баланса за два периода. Это помогает оценить динамику финансового состояния.

В Mini App также реализован экран прогноза расходов.



Рисунок 39 – Прогноз расходов

На рисунке 39 показан прогноз расходов. Система отображает ожидаемое значение, границы прогноза, тренд и краткое пояснение.

Для наглядности прогноз дополняется графиком динамики расходов по месяцам.



Рисунок 40 – Динамика расходов по месяцам и прогноз

На рисунке 40 показана динамика расходов. Пользователь может сравнить прошлые значения с прогнозом на следующий период.

Раздел бюджетов позволяет контролировать лимиты по категориям.



Рисунок 41 – Список бюджетов

На рисунке 41 показан список бюджетов. Для каждой категории отображается лимит, потраченная сумма, остаток и статус.

Кроме списка бюджетов, реализована общая сводка.

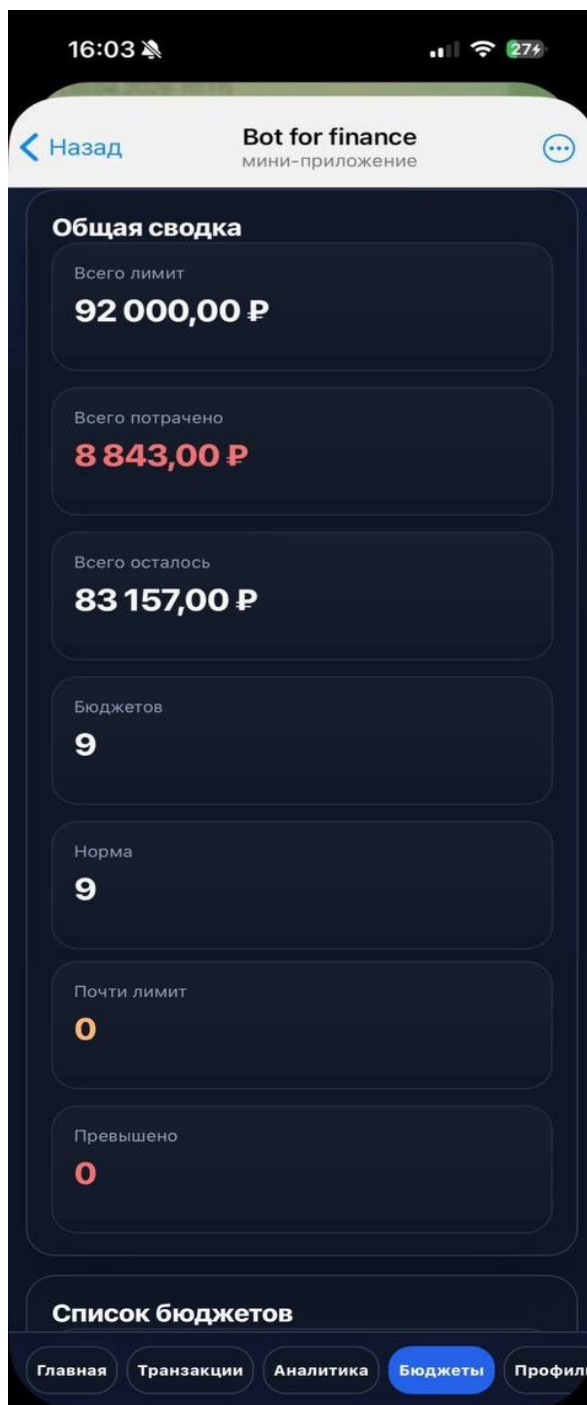


Рисунок 42 – Общая сводка по бюджетам

На рисунке 42 показана сводка по бюджетам. Она позволяет быстро оценить общий лимит, сумму расходов и оставшийся остаток.

Профиль пользователя в Mini App содержит персональные данные и финансовую сводку.

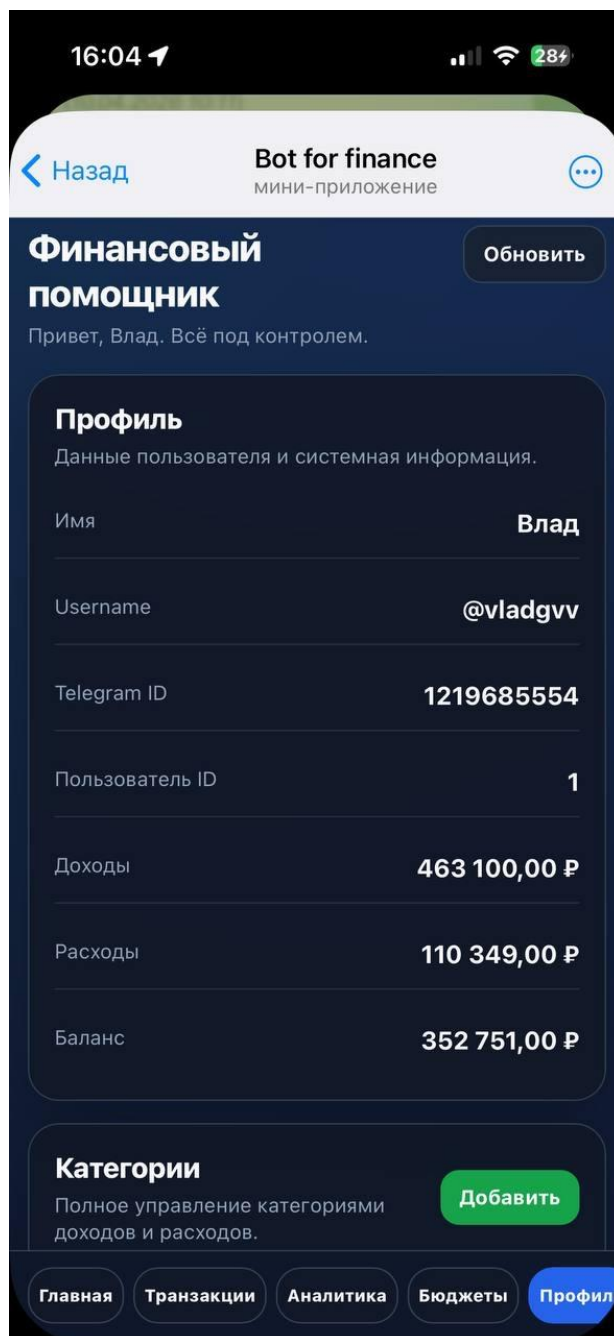


Рисунок 43 – Профиль пользователя в Mini App

На рисунке 43 показан профиль пользователя. В нем отображаются Telegram ID, username, внутренний идентификатор и финансовые показатели.

Для настройки структуры учета реализован экран управления категориями.

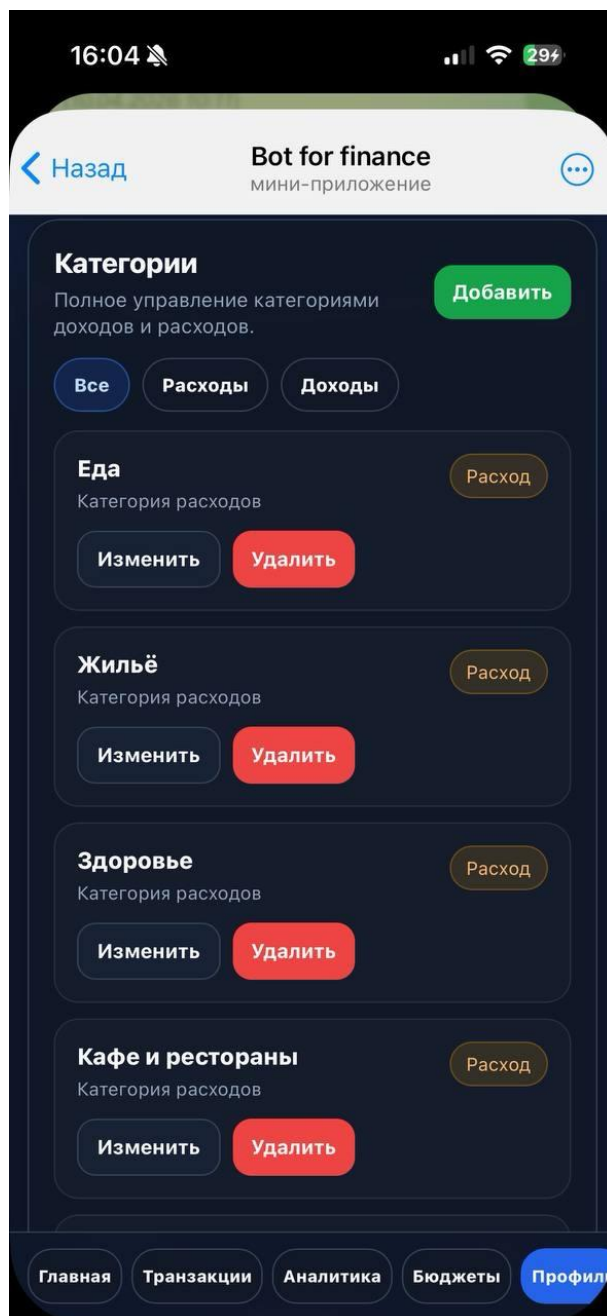


Рисунок 44 – Управление категориями

На рисунке 44 показан экран категорий. Пользователь может просматривать, изменять и удалять категории.

Для повторяющихся операций реализован раздел шаблонов.

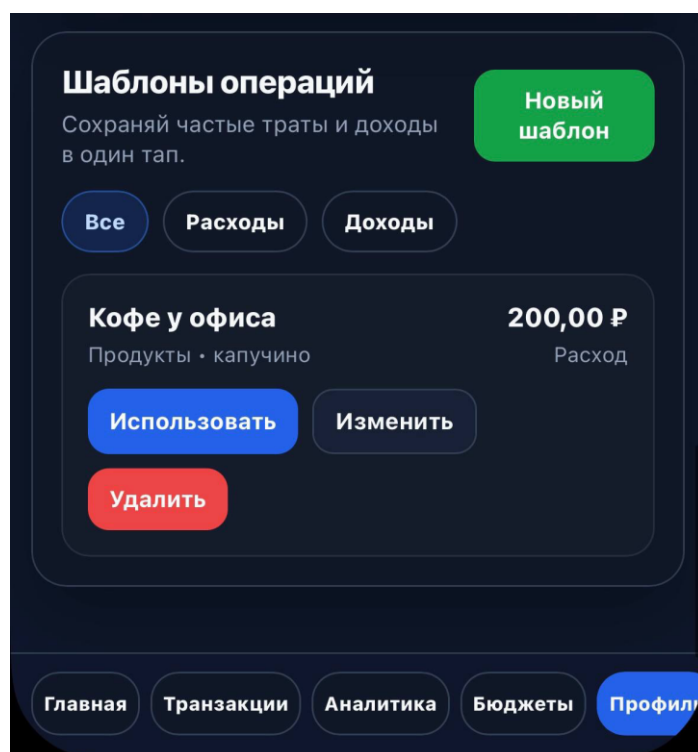


Рисунок 45 – Шаблоны операций

На рисунке 45 показан экран шаблонов. Он позволяет быстро использовать ранее сохраненные операции.

Таким образом, Mini App реализован как полноценный расширенный интерфейс. Он дополняет Telegram-бот и предоставляет пользователю удобную работу с данными, аналитикой, бюджетами и прогнозами.

3.6 Реализация REST API

REST API используется как связующий слой между Mini App, серверной логикой и базой данных. Через API веб-интерфейс получает данные о пользователе, транзакциях, категориях, бюджетах, шаблонах, аналитике и прогнозе [8], [9].

Основные обработчики API расположены в файле `web/finance/api.py`, а маршруты — в `web/finance/urls.py`. API разделен по группам: авторизация,

пользователь, категории, шаблоны, транзакции, быстрый ввод, сводка, аналитика, прогноз и бюджеты.

```
urlpatterns = [
    path(route: "health/", health),
    path(route: "miniapp/auth/", miniapp_auth),
    path(route: "me/", me),
    path(route: "categories/", categories),
    path(route: "categories/<int:category_id>/", category_detail),
    path(route: "templates/", templates),
    path(route: "templates/<int:template_id>/", template_detail),
    path(route: "templates/<int:template_id>/use/", use_template),
    path(route: "transactions/", transactions),
    path(route: "transactions/quick-add/preview/", quick_add_preview),
    path(route: "transactions/quick-add/create/", quick_add_create),
    path(route: "transactions/<int:tx_id>/", transaction_detail),
    path(route: "transactions/<int:tx_id>/repeat/", repeat_transaction),
    path(route: "summary/", summary),
    path(route: "analytics/overview/", analytics_overview),
    path(route: "analytics/series/", analytics_series),
    path(route: "forecast/", forecast_expenses),
    path(route: "insights/", insights),
    path(route: "budgets/", budgets),
    path(route: "budgets/<int:budget_id>/", budget_detail),
    path(route: "budgets/summary/", budgets_summary),
]
```

Рисунок 46 – Основные маршруты REST API

На рисунке 46 показаны основные маршруты REST API. Видно, что методы сгруппированы по сущностям системы.

Для Mini App реализована авторизация через данные Telegram. После проверки данных пользователь получает внутренний API-ключ, который используется в последующих запросах.

```

def miniapp_auth(request: Request) -> Response:
    init_data = (request.data.get("initData") or "").strip()
    bot_token = os.getenv("BOT_TOKEN", "").strip()

    if not init_data:
        return Response(
            data: {"detail": "Missing initData"},
            status=status.HTTP_400_BAD_REQUEST,
        )

    if not bot_token:
        return Response(
            data: {"detail": "BOT_TOKEN is not configured on server"},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR,
        )

    tg_user = _validate_telegram_init_data(init_data, bot_token)
    if not tg_user:
        return Response(
            data: {"detail": "Invalid Telegram initData"},
            status=status.HTTP_401_UNAUTHORIZED,
        )

    telegram_id = tg_user.get("id")
    if not telegram_id:
        return Response(
            data: {"detail": "Telegram user id not found"},
            status=status.HTTP_400_BAD_REQUEST,
        )

    username = tg_user.get("username")
    first_name = tg_user.get("first_name")

    user = User.objects.filter(telegram_id=telegram_id).first()

```

Рисунок 47 – Механизм авторизации Mini App через API

На рисунке 47 показана схема авторизации Mini App. Сначала проверяются данные Telegram, затем сервер выдает ключ доступа к данным пользователя.

Одной из основных частей API являются методы для работы с транзакциями. Через них Mini App получает список операций, создает новые записи, изменяет и удаляет существующие.

```

@api_view(["GET", "POST"])  Ⓜ Vladislav Gavrilov
def transactions(request: Request) -> Response:
    ⚡ user = get_user_from_request(request)

    if request.method == "GET":
        try:
            qs = _filtered_transactions_queryset(request, user).order_by(*field_names: "-date", "-id")
        except ValueError as exc:
            return Response(data={"detail": str(exc)}, status=status.HTTP_400_BAD_REQUEST)

        limit_raw = request.query_params.get("limit", "500")
        try:
            limit = max(1, min(int(limit_raw), 1000))
        except ValueError:
            return Response(
                data={"detail": "Некорректный параметр limit"},
                status=status.HTTP_400_BAD_REQUEST,
            )

        data = TransactionSerializer(qs[:limit], many=True).data
        return Response(data)

    tx_type = (request.data.get("type") or "").strip()
    if tx_type not in ("income", "expense"):
        return Response(
            data={"detail": "Поле type должно быть income или expense"},
            status=status.HTTP_400_BAD_REQUEST,
        )

```

Рисунок 48 – Фрагмент реализации методов REST API для работы с транзакциями

На рисунке 48 показан пример реализации методов для транзакций. Эти обработчики используются Mini App для работы с финансовыми операциями.

Для аналитики и прогнозирования API возвращает готовые агрегированные данные в формате JSON.

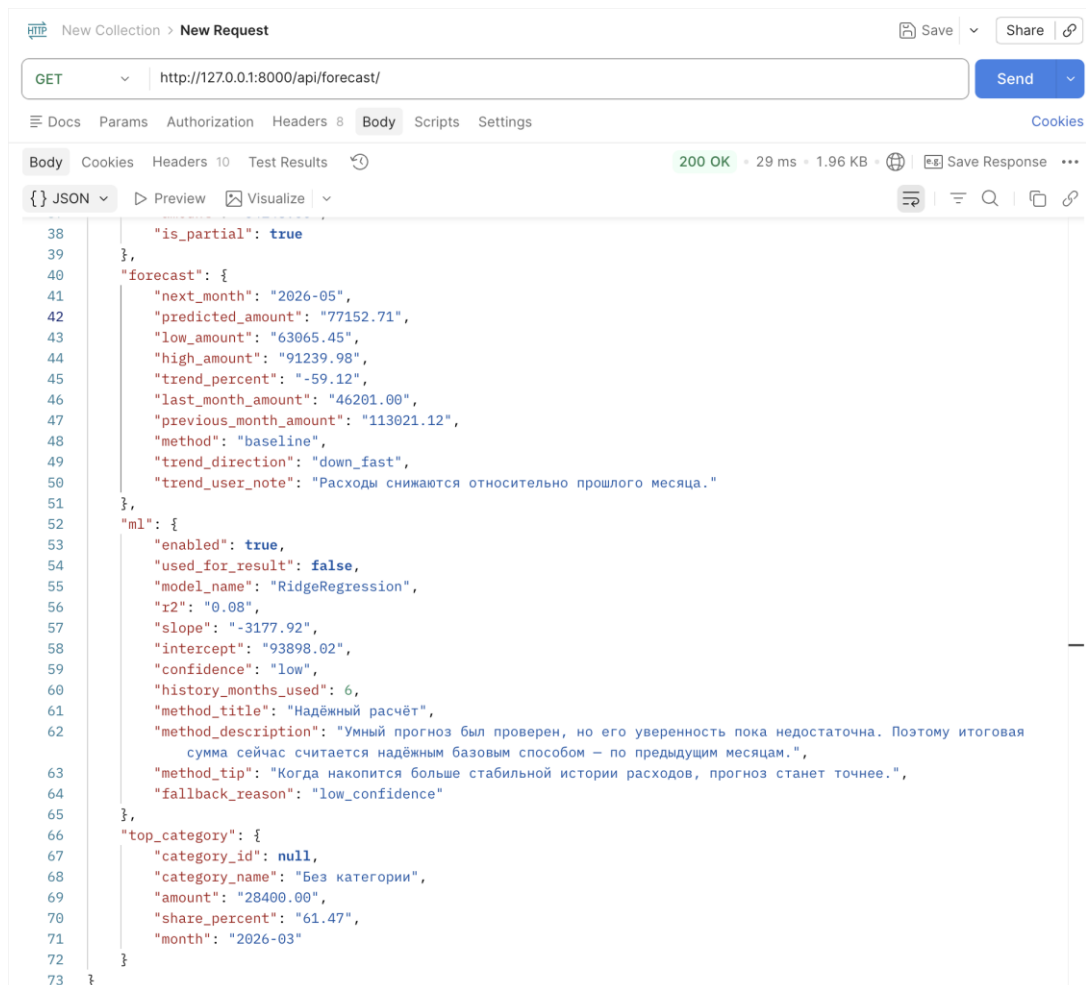


Рисунок 49 – Пример JSON-ответа REST API для прогнозирования расходов

На рисунке 49 показан пример JSON-ответа API. Сервер возвращает подготовленные данные, которые Mini App может сразу отобразить в интерфейсе.

Таким образом, REST API реализован как центральный механизм связи между Mini App и серверной частью. Он позволяет отделить интерфейс от бизнес-логики и использовать единые правила обработки данных.

3.7 Реализация аналитики и отчетов

Аналитика реализована на основе транзакций пользователя. Система рассчитывает доходы, расходы, баланс, количество операций, расходы по категориям, динамику по дням и сравнение периодов.

В Mini App графики строятся средствами Chart.js, так как визуализация отображается непосредственно в веб-интерфейсе. Matplotlib может использоваться для серверной генерации отдельных графиков и отчетных материалов [13]. В Mini App аналитические данные отображаются в виде диаграмм, карточек и сравнительных блоков. Для Telegram-бота аналитика выводится кратко: баланс, последние операции и основные показатели.

Экспорт отчетов реализован в формате Excel. Для этого используются openpyxl и XlsxWriter [14], [15]. Пользователь может выгрузить все операции или данные за конкретный месяц.

3.8 Реализация машинного обучения и прогнозирования

Модуль машинного обучения используется для автоматической категоризации расходов. При быстром вводе расхода система получает описание операции и сумму, после чего пытается определить подходящую категорию.

Обработка выполняется в несколько этапов. Сначала текст нормализуется: приводится к нижнему регистру, очищается от лишних символов и подготавливается для анализа. Затем применяется проверка по ключевым словам. Если категория не определена правилами, используется модель машинного обучения.

Для преобразования текста используется TF-IDF-векторизация [11]. Для классификации применяется логистическая регрессия [12]. Такой подход подходит для задачи определения категории по короткому текстовому описанию операции.

Результат работы ML-модуля не сохраняется без контроля пользователя. Бот предлагает категорию и показывает уверенность, а пользователь подтверждает ее или выбирает другую. Это делает систему удобнее и одновременно снижает риск ошибочной категоризации.

Прогнозирование расходов реализовано на основе истории транзакций. Система анализирует расходы за предыдущие месяцы, определяет динамику и рассчитывает предполагаемое значение на следующий период. Если данных недостаточно, используется более простой расчет по последним значениям.

```
ARTIFACTS_DIR = os.path.join(os.path.dirname(__file__), "artifacts")

with open(os.path.join(ARTIFACTS_DIR, "model.pkl"), "rb") as f:
    model = pickle.load(f)

with open(os.path.join(ARTIFACTS_DIR, "vectorizer.pkl"), "rb") as f:
    vectorizer = pickle.load(f)

LABEL_MAP_PATH = os.path.join(ARTIFACTS_DIR, "label_map.pkl")
label_map = None
if os.path.exists(LABEL_MAP_PATH):
    with open(LABEL_MAP_PATH, "rb") as f:
        label_map = pickle.load(f)
```

Рисунок 50 – Реализация автоматической категоризации расходов

На рисунке 50 показан фрагмент реализации автоматической категоризации. Модуль помогает пользователю быстрее выбирать категорию расхода.



Рисунок 51 – Отображение прогноза расходов

На рисунке 51 показано отображение прогноза расходов. Пользователь видит ожидаемую сумму и дополнительную информацию для планирования бюджета.

3.9 Реализация дополнительных функций

К дополнительным функциям системы относятся импорт банковских выписок, экспорт отчетов, напоминания, шаблоны операций и административные возможности.

Импорт выписок реализован для PDF-файлов. Пользователь выбирает банк, отправляет файл, после чего система извлекает операции и показывает предварительный просмотр. Для обработки PDF используется pdfplumber [16].

Экспорт позволяет сформировать Excel-файл с финансовыми операциями. Это удобно для хранения отчета или дальнейшей работы с данными вне системы.

Напоминания помогают пользователю регулярно вести учет. Система проверяет наличие операций за текущий день и при необходимости отправляет уведомление.

Шаблоны операций ускоряют добавление повторяющихся доходов и расходов. Пользователь может сохранить типовую операцию и затем использовать ее повторно.

Административные функции используются для сопровождения системы: просмотра пользователей, статистики, крупных транзакций, экспорта данных и удаления записей при необходимости.

Для сопровождения системы также используется административная панель Django. Она позволяет администратору просматривать и изменять основные сущности системы без прямого обращения к базе данных. Через панель доступны пользователи, категории, транзакции, бюджеты, коды авторизации и шаблоны транзакций. Это упрощает контроль данных и обслуживание системы при тестировании и эксплуатации [7].

Панель управления

FINANCE

Бюджеты	+ Добавить	✎ Изменить
Категории	+ Добавить	✎ Изменить
Коды авторизации	+ Добавить	✎ Изменить
Пользователи	+ Добавить	✎ Изменить
Транзакции	+ Добавить	✎ Изменить
Шаблоны транзакций	+ Добавить	✎ Изменить

Последние действия

Мои действия

- ✗ Прочие доходы (income)
Категория
- ✗ транспорт (income)
Категория
- ✗ ЗП (income)
Категория
- ✗ aa (expense)
Категория
- ✗ Транспорт (expense)
Категория
- ✗ income: 750.00 — Прочие
доходы (2026-02-05)
Транзакция
- ✗ expense: 660.00 — Прочее
(2026-02-05)
Транзакция
- ✗ income: 3182.00 — Прочие
доходы (2026-02-28)
Транзакция
- ✗ income: 353.00 — Прочие
доходы (2026-02-28)
Транзакция
- ✗ income: 5000.00 — Прочие
доходы (2026-03-06)
Транзакция

Рисунок 52 – Административная панель системы

На рисунке 52 показана административная панель системы. В ней отображаются основные разделы базы данных: бюджеты, категории, пользователи, транзакции, коды авторизации и шаблоны транзакций. Справа отображается список последних действий администратора, что позволяет контролировать изменения, выполненные через панель управления.

3.10 Выводы по разделу

В третьем разделе была описана реализация программного обеспечения. Были рассмотрены выбранные средства разработки, структура базы данных, организация проекта, Telegram-бот, Mini App, REST API, аналитика, машинное обучение, прогнозирование и дополнительные функции.

В реализации сохранены все основные пользовательские сценарии: быстрый ввод операций, просмотр баланса и истории, работа с Mini App,

аналитика, бюджеты, импорт выписок, экспорт отчетов, напоминания, шаблоны и автоматическая категоризация расходов. Система реализована как единое клиент-серверное решение, где Telegram-бот и Mini App являются пользовательскими интерфейсами, REST API обеспечивает обмен данными, а PostgreSQL используется как основное хранилище.

4 Тестирование и оценка результатов

4.1 Цель и методы тестирования

После реализации системы было проведено тестирование основных компонентов: Telegram-бота, Mini App, REST API, базы данных, аналитики, модуля машинного обучения, импорта, экспорта, напоминаний и административной панели.

Цель тестирования заключалась в проверке того, что система корректно выполняет основные пользовательские сценарии: добавление доходов и расходов, просмотр истории, расчет баланса, отображение аналитики, работа с бюджетами, автоматическая категоризация расходов, прогнозирование и обмен данными между Mini App и серверной частью.

В работе использовались следующие виды тестирования:

- функциональное тестирование;
- интеграционное тестирование;
- тестирование пользовательских сценариев;
- тестирование REST API через Postman [17];
- проверка обработки некорректных данных;
- ручная проверка интерфейсов Telegram-бота и Mini App;
- контрольная проверка аналитических расчетов;
- проверка работы ML-категоризации и прогнозирования.
- Такой подход позволяет проверить не только отдельные функции, но и работу системы как единого программного продукта [20].

4.2 Тестирование Telegram-бота

Telegram-бот тестировался как основной интерфейс быстрого ввода операций. Проверялись команды, кнопки, быстрый текстовый ввод,

добавление доходов и расходов, просмотр баланса, история операций, импорт выписки, экспорт отчета, напоминания и административные функции.

Таблица 2 – Результаты тестирования Telegram-бота

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
1	Запуск команды /start	Пользователь регистрируется, открывается стартовое меню	Выполнено успешно
2	Открытие главного меню	Отображаются основные кнопки системы	Выполнено успешно
3	Быстрый ввод дохода	Доход распознается и сохраняется в БД	Выполнено успешно
4	Быстрый ввод расхода	Расход распознается, предлагается категория	Выполнено успешно
5	Подтверждение ML-категории	Категория сохраняется вместе с транзакцией	Выполнено успешно
6	Ручной выбор категории	Пользователь может изменить предложенную категорию	Выполнено успешно
7	Просмотр баланса	Бот выводит доходы, расходы и итоговый баланс	Выполнено успешно
8	Просмотр последних операций	Отображается список последних транзакций	Выполнено успешно
9	Экспорт в Excel	Формируется и отправляется Excel-файл	Выполнено успешно
10	Импорт выписки	Бот принимает файл и показывает найденные операции	Выполнено успешно
11	Настройка напоминаний	Пользователь может включить или отключить уведомления	Выполнено успешно
12	Административные команды	Администратор получает доступ к служебным функциям	Выполнено успешно

Проверка показала, что Telegram-бот корректно выполняет основные сценарии. Быстрый ввод работает как для доходов, так и для расходов, а при

добавлении расхода система предлагает категорию автоматически. Это снижает количество ручных действий пользователя.

4.3 Тестирование Mini App

Mini App тестировался как расширенный интерфейс для работы с финансовыми данными. Основное внимание уделялось отображению транзакций, фильтрации, аналитике, бюджетам, категориям, шаблонам и прогнозу расходов.

Таблица 3 – Результаты тестирования Mini App

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
1	Открытие Mini App из Telegram	Загружается главный экран приложения	Выполнено успешно
2	Получение данных пользователя	Отображается профиль и финансовая сводка	Выполнено успешно
3	Просмотр транзакций	Загружается список операций пользователя	Выполнено успешно
4	Фильтрация транзакций	Отображаются операции по выбранным параметрам	Выполнено успешно
5	Добавление операции	Новая транзакция сохраняется в базе данных	Выполнено успешно
6	Редактирование операции	Изменения сохраняются и отображаются в интерфейсе	Выполнено успешно
7	Удаление операции	Операция удаляется из списка и базы данных	Выполнено успешно
8	Открытие аналитики	Отображаются сводки и графики	Выполнено успешно
9	Открытие бюджетов	Показываются лимиты, расходы и остатки	Выполнено успешно
10	Использование шаблона	Создается операция на основе шаблона	Выполнено успешно
11	Открытие прогноза	Отображается прогноз расходов	Выполнено успешно
12	Управление категориями	Категории отображаются и редактируются	Выполнено успешно

Тестирование подтвердило, что Mini App удобно использовать для более подробной работы с данными. Если Telegram-бот подходит для быстрого ввода, то Mini App лучше подходит для анализа, фильтрации, просмотра бюджетов и работы с прогнозами.

4.4 Тестирование REST API

REST API является связующим компонентом между Mini App, серверной частью и базой данных. Проверка выполнялась через Postman и через реальные запросы со стороны Mini App [17].

Тестировались успешные и ошибочные сценарии: корректная авторизация, отсутствие API-ключа, неверные параметры, создание транзакции, быстрый ввод, получение аналитики и прогноза.

Таблица 4 – Результаты тестирования REST API

№	Endpoint / сценарий	Ожидаемый результат	Фактический результат
1	GET /api/health/	Возвращается статус доступности сервиса	Статус 200
2	POST /api/miniapp/auth/ с корректными данными	Пользователь авторизуется	Выполнено успешно
3	POST /api/miniapp/auth/ с ошибочными данными	Доступ запрещается	Статус 401
4	GET /api/me/ с API-ключом	Возвращаются данные пользователя	Выполнено успешно
5	GET /api/transactions/	Возвращается список транзакций	Выполнено успешно
6	POST /api/transactions/	Создается новая транзакция	Статус 201
7	POST /api/transactions/ без обязательных полей	Возвращается ошибка валидации	Статус 400
8	POST /api/transactions/quick-add/preview/	Возвращается разобранная операция	Выполнено успешно
9	GET /api/analytics/overview/	Возвращается финансовая сводка	Выполнено успешно

10	GET /api/forecast/	Возвращается прогноз расходов	Выполнено успешно
11	GET /api/budgets/summary/	Возвращается сводка по бюджетам	Выполнено успешно

Проверка показала, что REST API корректно обрабатывает запросы, возвращает данные в формате JSON и отделяет клиентский интерфейс от серверной логики [8], [9].

4.5 Тестирование аналитики

Аналитика проверялась через сравнение данных в интерфейсе с контрольными расчетами по транзакциям. Проверялись сумма доходов, сумма расходов, баланс, расходы по категориям, расходы по дням и сравнение периодов.

Таблица 5 – Проверка аналитических расчетов

№	Проверяемый показатель	Способ проверки	Результат
1	Общая сумма доходов	Сравнение с суммой income-транзакций	Корректно
2	Общая сумма расходов	Сравнение с суммой expense-транзакций	Корректно
3	Баланс	Доходы минус расходы	Корректно
4	Расходы по категориям	Группировка expense-транзакций по категории	Корректно
5	Расходы по дням	Группировка операций по дате	Корректно
6	Сравнение периодов	Сравнение текущего и предыдущего периода	Корректно
7	Бюджеты	Сравнение лимита и фактических расходов	Корректно

Результаты проверки показали, что аналитический модуль правильно обрабатывает данные из базы и формирует показатели для Mini App. Графики и сводные карточки соответствуют фактическим данным пользователя.

Для дополнительной проверки был выполнен контрольный расчет. Например, при доходе 50000 рублей и расходах 3000, 1500 и 700 рублей ожидаемая сумма расходов составляет 5200 рублей, а баланс — 44800 рублей. Полученные системой значения совпали с ручным расчетом.

4.6 Тестирование ML-категоризации и прогнозирования

Модуль машинного обучения тестировался на примерах пользовательского ввода. Проверялась способность системы определить категорию расхода по описанию операции и сумме.

Таблица 6 – Проверка автоматической категоризации расходов

№	Ввод пользователя	Ожидаемая категория	Результат системы	Итог
1	-500 бензин	Транспорт	Транспорт	Успешно
2	-1200 продукты	Продукты	Продукты	Успешно
3	-399 подписка	Подписки	Подписки	Успешно
4	-250 кофе	Кафе / Продукты	Предложена подходящая категория	Успешно
5	-3000 одежда	Одежда	Одежда	Успешно
6	-700 такси	Транспорт	Транспорт	Успешно

При категоризации система сначала использует проверку по ключевым словам, а затем при необходимости применяет ML-модель. Для обработки текста используется TF-IDF-векторизация, а для классификации — логистическая регрессия [10], [11], [12].

Важной особенностью является то, что пользователь не теряет контроль над результатом. Если предложенная категория подходит, он подтверждает ее. Если система ошиблась, пользователь выбирает категорию вручную. Это делает работу модуля безопаснее и удобнее.

Прогнозирование проверялось на истории расходов пользователя за предыдущие месяцы. Система анализирует динамику, рассчитывает ожидаемую сумму расходов и отображает результат в Mini App.

Таблица 7 – Проверка прогнозирования расходов

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
1	Наличие истории расходов	Система строит прогноз	Выполнено успешно
2	Недостаточно данных	Используется упрощенный расчет	Выполнено успешно
3	Расходы растут	Прогноз показывает увеличение	Выполнено успешно
4	Расходы снижаются	Прогноз показывает снижение	Выполнено успешно
5	Открытие прогноза в Mini App	Отображаются сумма, диапазон и пояснение	Выполнено успешно

Тестирование показало, что ML-модуль помогает ускорить ввод расходов, а прогнозирование дает пользователю дополнительную информацию для планирования бюджета.

4.7 Тестирование импорта, экспорта и дополнительных функций

Дополнительно проверялись функции импорта выписок, экспорта отчетов, напоминаний, шаблонов и административной панели.

Таблица 8 – Тестирование дополнительных функций

№	Функция	Ожидаемый результат	Фактический результат
1	Импорт PDF-выписки	Система извлекает операции из файла	Выполнено успешно
2	Предварительный просмотр импорта	Пользователь видит найденные операции	Выполнено успешно
3	Сохранение импортированных операций	Операции добавляются в базу данных	Выполнено успешно
4	Экспорт в Excel	Формируется файл с операциями	Выполнено успешно
5	Напоминания	Пользователь получает уведомление	Выполнено успешно

6	Шаблоны операций	Новая операция создается по шаблону	Выполнено успешно
7	Административная панель	Администратор видит основные сущности системы	Выполнено успешно

Проверка дополнительных функций показала, что система поддерживает не только ручной ввод операций, но и более удобные сценарии работы: импорт банковских данных, выгрузку отчетов, регулярные напоминания и администрирование.

4.8 Анализ результатов тестирования

По результатам тестирования можно сделать вывод, что разработанная система выполняет основные требования, сформулированные в первом разделе. Telegram-бот корректно работает с быстрым вводом, Mini App отображает расширенный интерфейс, REST API обеспечивает обмен данными, а PostgreSQL используется как центральное хранилище.

Таблица 9 – Итоговая оценка результатов тестирования

Компонент	Что проверялось	Результат
Telegram-бот	Ввод операций, баланс, история, импорт, экспорт	Работает корректно
Mini App	Транзакции, аналитика, бюджеты, прогноз, категории	Работает корректно
REST API	Авторизация, транзакции, аналитика, прогноз	Работает корректно
База данных	Сохранение и получение данных	Работает корректно
Аналитика	Доходы, расходы, баланс, графики	Работает корректно
ML-модуль	Категоризация расходов	Работает корректно
Прогнозирование	Расчет будущих расходов	Работает корректно
Импорт и экспорт	PDF-выписки и Excel-отчеты	Работает корректно

Админ-панель	Просмотр и изменение сущностей	Работает корректно
--------------	--------------------------------	--------------------

В ходе тестирования были проверены как успешные, так и ошибочные сценарии. Система корректно обрабатывает отсутствие обязательных данных, неверный формат быстрого ввода, ошибки авторизации и некорректные параметры запросов.

Разработанная система может использоваться как рабочий инструмент для учета личных финансов. Она объединяет быстрый ввод через Telegram-бота, расширенный интерфейс Mini App, хранение данных в PostgreSQL, REST API, аналитику, бюджетирование и машинное обучение.

4.9 Выводы по разделу

В четвертом разделе было проведено тестирование разработанной системы. Проверка охватила Telegram-бота, Mini App, REST API, базу данных, аналитику, ML-категоризацию, прогнозирование, импорт, экспорт, напоминания, шаблоны и административную панель.

Результаты тестирования показали, что основные функции системы работают корректно. Пользователь может добавлять доходы и расходы, просматривать историю и баланс, работать с Mini App, анализировать расходы, контролировать бюджеты, импортировать выписки, экспортировать отчеты и получать прогноз расходов.

Таким образом, цель разработки была достигнута: создана единая интеллектуальная система учета, анализа и прогнозирования личных финансов на основе Telegram-бота и Mini App.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана интеллектуальная система учета, анализа и прогнозирования личных финансов на основе Telegram-бота и Telegram Mini App:

1. Был проведен анализ предметной области учета личных финансов. Рассмотрены основные способы ведения учета: таблицы, банковские приложения, мобильные приложения и Telegram-боты. Также были изучены аналоги @coountrybot и @mobs_bot, выявлены их преимущества и ограничения.
2. На основе анализа были сформулированы требования к разрабатываемой системе. Были определены функциональные, нефункциональные, структурные, программные и защитные требования. Основное внимание было уделено удобству быстрого ввода операций, хранению данных, аналитике, бюджетированию и применению машинного обучения.
3. Была спроектирована архитектура системы. В состав системы вошли Telegram-бот, Telegram Mini App, серверная часть, REST API, база данных PostgreSQL, модуль аналитики, модуль машинного обучения, модуль прогнозирования, импорт выписок, экспорт отчетов, напоминания и административная панель.
4. Была разработана структура базы данных. В ней предусмотрено хранение пользователей, категорий, транзакций, бюджетов, напоминаний, шаблонов операций, данных импорта и прогнозных значений. Такая структура позволяет хранить основные и дополнительные данные системы в связанном виде.
5. Был реализован Telegram-бот, который используется для быстрого взаимодействия с пользователем. Через него можно добавлять доходы и расходы, просматривать баланс и историю, импортировать выписки,

- экспортировать данные, настраивать напоминания и открывать Mini App.
6. Был реализован Telegram Mini App. Он используется как расширенный интерфейс для работы с финансовыми данными. В Mini App доступны транзакции, аналитика, бюджеты, категории, шаблоны, профиль пользователя и прогноз расходов.
 7. Был реализован REST API, обеспечивающий обмен данными между Mini App, серверной частью и базой данных. Через API выполняются авторизация, работа с транзакциями, категориями, бюджетами, шаблонами, аналитикой и прогнозами.
 8. В системе реализованы аналитические функции. Пользователь может просматривать сумму доходов, сумму расходов, баланс, структуру расходов по категориям, динамику по дням, сравнение периодов и состояние бюджетов.
 9. Была реализована автоматическая категоризация расходов с использованием методов машинного обучения. Система анализирует описание операции и сумму, после чего предлагает подходящую категорию. Пользователь может подтвердить предложенный вариант или выбрать категорию вручную.
 10. Был реализован модуль прогнозирования расходов. Он использует историю транзакций пользователя и формирует прогноз будущих расходов. Это помогает пользователю заранее оценивать возможные траты и планировать бюджет.
 11. Также были реализованы дополнительные функции: импорт банковских выписок, экспорт отчетов в Excel, шаблоны операций, напоминания и административная панель. Эти возможности делают систему более удобной для практического использования.
 12. Проведенное тестирование показало, что основные функции системы работают корректно. Были проверены Telegram-бот, Mini App, REST

API, база данных, аналитика, ML-категоризация, прогнозирование, импорт, экспорт и административные функции.

13. Таким образом, цель выпускной квалификационной работы была достигнута. В результате была разработана программная система, которая объединяет быстрый ввод операций через Telegram-бота, расширенную работу с данными через Mini App, хранение информации в PostgreSQL, аналитику, бюджетирование и элементы машинного обучения. Разработанная система может использоваться как практический инструмент для учета, анализа и прогнозирования личных финансов.
14. В дальнейшем систему можно развивать за счет добавления мультивалютного учета, расширения списка поддерживаемых банковских выписок, улучшения качества ML-моделей, добавления более сложных методов прогнозирования и реализации веб-кабинета вне Telegram.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Митрофанова, Е. Ю. Методические указания по оформлению выпускных работ бакалавров / Е. Ю. Митрофанова, А. А. Сирота. – Воронеж : ВГУ, 2016. – Текст : непосредственный.
2. Telegram Bot API. – Текст : электронный // Telegram Core. – URL : <https://core.telegram.org/bots/api> (дата обращения : 28.04.2026).
3. Telegram Mini Apps. – Текст : электронный // Telegram Core. – URL : <https://core.telegram.org/bots/webapps> (дата обращения : 28.04.2026).
4. aiogram Documentation. – Текст : электронный // aiogram. – URL : <https://docs.aiogram.dev/> (дата обращения : 28.04.2026).
5. Python Documentation. – Текст : электронный // Python Software Foundation. – URL : <https://docs.python.org/3/> (дата обращения : 28.04.2026).
6. PostgreSQL Documentation. – Текст : электронный // PostgreSQL. – URL : <https://www.postgresql.org/docs/> (дата обращения : 28.04.2026).
7. Django Documentation. – Текст : электронный // Django Software Foundation. – URL : <https://docs.djangoproject.com/> (дата обращения : 28.04.2026).
8. Django REST Framework Documentation. – Текст : электронный // Django REST Framework. – URL : <https://www.django-rest-framework.org/> (дата обращения : 28.04.2026).
9. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures : doctoral dissertation / R. T. Fielding. – Irvine : University of California, 2000. – Текст : электронный. – URL : <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (дата обращения : 28.04.2026).
10. scikit-learn User Guide. – Текст : электронный // scikit-learn. – URL : https://scikit-learn.org/stable/user_guide.html (дата обращения : 28.04.2026).
11. TfidfVectorizer. – Текст : электронный // scikit-learn. – URL : [https://scikit-](https://scikit-learn.org/stable/user_guide.html)

learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (дата обращения : 28.04.2026).

12. LogisticRegression. – Текст : электронный // scikit-learn. – URL : https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (дата обращения : 28.04.2026).

13. Matplotlib Documentation. – Текст : электронный // Matplotlib. – URL : <https://matplotlib.org/stable/> (дата обращения : 28.04.2026).

14. openpyxl Documentation. – Текст : электронный // openpyxl. – URL : <https://openpyxl.readthedocs.io/> (дата обращения : 28.04.2026).

15. XlsxWriter Documentation. – Текст : электронный // XlsxWriter. – URL : <https://xlsxwriter.readthedocs.io/> (дата обращения : 28.04.2026).

16. pdfplumber Documentation. – Текст : электронный // GitHub. – URL : <https://github.com/jsvine/pdfplumber> (дата обращения : 28.04.2026).

17. Postman Documentation. – Текст : электронный // Postman Learning Center. – URL : <https://learning.postman.com/docs/> (дата обращения : 28.04.2026).

18. Sommerville, I. Software Engineering / I. Sommerville. – 10th ed. – Boston : Pearson, 2015. – Текст : непосредственный.

19. Date, C. J. An Introduction to Database Systems / C. J. Date. – 8th ed. – Boston : Pearson, 2003. – Текст : непосредственный.

20. Myers, G. J. The Art of Software Testing / G. J. Myers, C. Sandler, T. Badgett. – 3rd ed. – Hoboken : Wiley, 2011. – Текст : непосредственный.

21. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow / A. Géron. – 3rd ed. – Sebastopol : O'Reilly Media, 2022. – Текст : непосредственный.

22. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – Текст : непосредственный.

23. ГОСТ 7.1–2003. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : ИПК Издательство стандартов, 2004. – Текст : непосредственный.